Aula 03 - Prof. Vinícius Borges e Raphael Lacerda

SEFAZ-AL (Auditor Fiscal da Administração Tributária Estadual - AFTE) Desenvolvimento de Sistemas - 2026 (Pós-Edital)

Autor:
Vinicius Borges

27 de Agosto de 2026

02648216103 - HENRIQUE AMORIM BARRETO

Índice

1) GIT - Teoria 3

2) GIT - Questões Comentadas 36

3) GIT - Lista de Questões 45

4) Gitlab - Teoria 52

5) Gitlab - Questões Comentadas 71

6) Gitlab - Lista de Questões 85

GIT

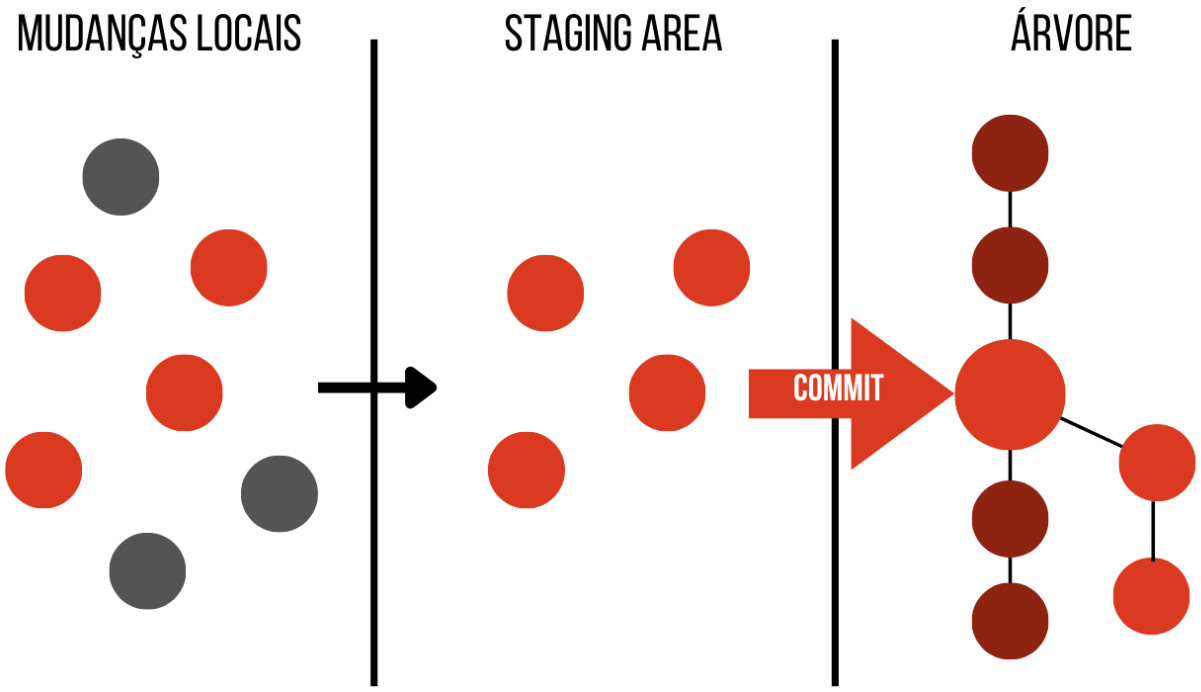
Conceitos Básicos

Git é uma ferramenta *open source* desenvolvida para controle de versionamento de códigos, criada originalmente como uma ferramenta nativa ao Linux. Em 2005, ganhou destaque pelo seu foco no desenvolvimento do kernel do Linux. Atualmente, o *Git* possui portabilidade com diversos sistemas operacionais, além de funcionar nativamente em algumas IDEs – como o Visual Studio Code.

A grande diferença do Git para outros VCS é a forma como os dados são abordados – enquanto a maioria dos VCS armazenam informações como uma lista de mudanças, o Git aborda os dados como uma série de *snapshots*, fotografias do estado atual do sistema.

O fluxo de trabalho do Git é simples:

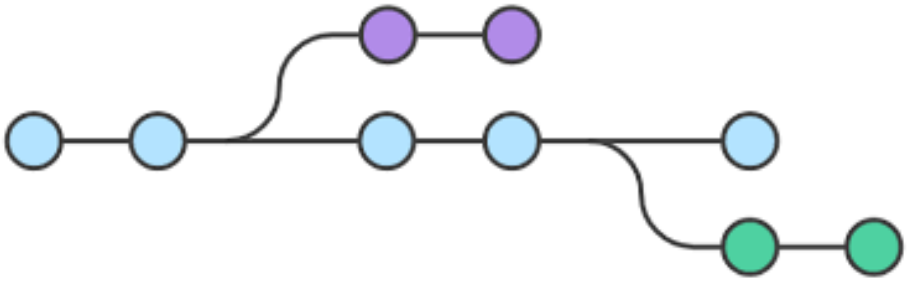
- Você modifica um arquivo na sua árvore de trabalho (veremos mais à frente sobre isso) – colocando-os no estado *modified*.
- Você seleciona quais mudanças serão aplicadas ao repositório principal, encaminhando-as para uma área chamada *staging area* – colocando-os no estado *staged*.
- Você confirma as mudanças através de um *commit*, e as mudanças da *staging area* são armazenadas no repositório principal – colocando-os no estado *commited*.



Podemos usar os comandos do Git de diversas formas – formas mais interativas, com interfaces de usuário (GUI), ou formas mais técnicas, através de interfaces de linha de comando (CLI). Para essa aula, usaremos a abordagem de comandos via CLI, já que é o que as bancas costumam cobrar.

Árvore de Trabalho

A árvore de trabalho (**working tree**) é uma forma de visualizar e controlar as diferentes versões do trabalho. Ela é o diretório do sistema de arquivos, é onde você edita, adiciona ou remove versões. Imagine cada versão como um nó nessa árvore – vamos da versão 0 para a versão 1, avançamos um nó; podemos ter um nó paralelo, com outras funcionalidades, e assim por diante. O relacionamento entre esses nós cria uma figura de árvore.



Uma árvore irá reunir todos os *commits*, fornecerá uma visualização de todas as versões anteriores, permitirá ir e voltar em versões diferentes, seguir galhos diferentes, e ai por diante. É, basicamente, a espinha dorsal do nosso sistema.

Fluxo de Trabalho

Digamos que você esteja trabalhando em um código, e deseja iniciar um repositório Git na sua máquina local – para acompanhar, por exemplo, os arquivos que fizemos na aula de Docker. Para isso, você irá usar o comando de inicialização:

```
git init
```

Pode ser que você queira puxar um repositório da internet pronto, para trabalhar em mudanças localmente em algo que já exista. Para isso, usamos a ferramenta de clone:

```
git clone
```

Ok, criamos o repositório, e fizemos mudanças. Quando fazemos essas mudanças, primeiramente precisamos passá-las para a *staging area*, uma área intermediária entre as mudanças locais e o repositório. Esse espaço é ideal para criarmos uma separação e não alterarmos diretamente o repositório.

Para adicionarmos os arquivos a essa área, usamos o comando *add*:

```
git add <nome-do-arquivo>
```


02648216103 - HENRIQUE AMORIM BARRETO

Por fim, quando quisermos confirmar essas mudanças e juntá-las ao repositório central, usamos o comando de confirmação, acompanhado de uma mensagem para identificar essa confirmação.

```
git commit -m “Mensagem do commit”
```

E pronto, você aprendeu Git rs brincadeiras à parte, essa é a estrutura central do fluxo de trabalho do Git. Agora, vamos aprender como são os componentes, como trabalhar com diferentes versões ao mesmo tempo, e utilizar o Git em todo o seu potencial de versionamento.

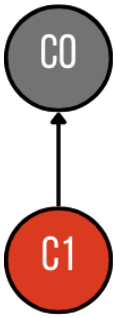
Ao longo de toda a aula, iremos “trabalhar” numa mesma árvore de trabalho para que você possa visualizar o que acontece. Vamos criar um repositório com o *git init*, e criar nosso primeiro *commit*, o C0, nele.

```
git init
git commit -m “Commit C0”
```



Vamos começar nossa árvore, dando um *commit* e criando C1, após determinada mudança, no nosso repositório inicial C0.

```
git commit -m “Commit C1”
```



(VUNESP/UNICAMP/2023) Dentre os diversos comandos utilizados no Git, o comando responsável por criar um novo repositório do Git é o:

a) git new
b) git mkdir
c) git init
d) git start
e) git begin

Comentários:

Temos dois comandos que podem criar repositórios locais – o *git init* e o *git clone*. Nesse sentido, correto o apontamento da letra C. *(Gabarito: Letra C)*

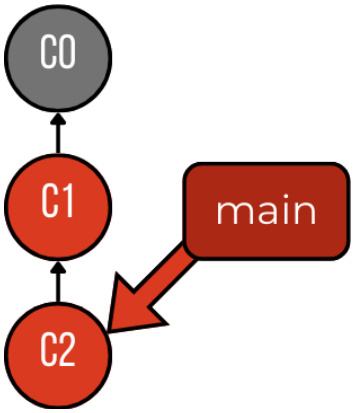
Branches

Quando criamos um *commit*, estamos, na verdade, criando um ponteiro no repositório central que aponta para a fotografia que tiramos do sistema naquele momento. Esse *commit* também tem ponteiros que apontam para as versões anteriores, e eventuais versões posteriores que são adicionadas.

Com esse emaranhado de ponteiros, começamos a formar nossa árvore. A árvore será composta por diversos ramos – ou, em inglês, como referenciaremos a partir de agora, ***branches***. Cada *branch* cria uma nova "espinha dorsal" dentro do nosso projeto. Ficará mais fácil de entender conforme vamos avançando no nosso projeto, e criando novas ramificações para implementarmos novas funcionalidades.

O *branch* principal é chamado de ***main branch***. Atualmente, no nosso projeto, estamos trabalhando nessa ramificação. Vamos fazer mais um *commit* agora, para aumentar o tamanho dessa espinha dorsal central. Vamos criar o *commit* C2.

```
git commit -m "Commit C2"
```



Porém, pode ser que queiramos trabalhar em uma funcionalidade diferente, uma versão diferente, testar cores, enfim, algum desenvolvimento de forma paralela ao desenvolvimento do "geral" da aplicação. Para isso, criamos **ramificações** – os nossos *branches*.

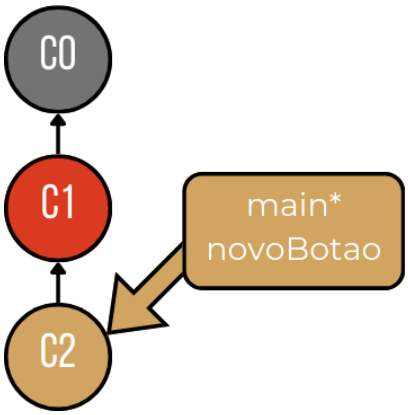
O Git tem um ditado: **ramifique cedo**, e **ramifique sempre**. Como não temos memória de armazenamento no Git associada à criação de ramos, é mais fácil dividirmos logicamente o

trabalho para organizar as versões. Iremos entender melhor os ramos e os *commits* ao longo da aula, mas grave uma coisa – um ramo inclui todos os trabalhos do *commit* e de seus ancestrais. Para criar um ramo novo, usamos o comando:

```
git branch <nome-do-ramo>
```

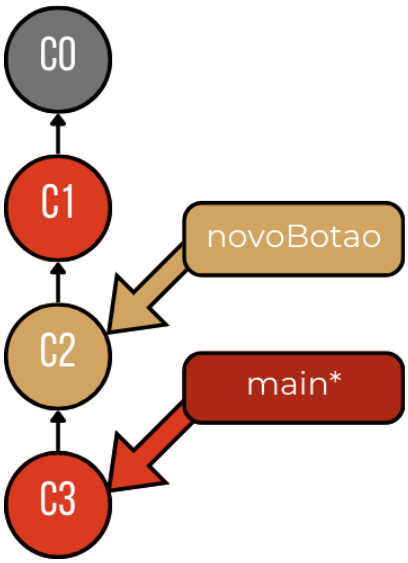
Vamos criar um ramo para uma nova funcionalidade – chamaremos esse ramo de novoBotao.

```
git branch novoBotao
```



Pronto, criamos o nosso *branch*. Mas perceba que ele coexiste na mesma posição do nosso *branch main*. Além disso, nós só criamos o *branch*, não mudamos para ele (*a posição que estamos é indicada pelo asterisco ao lado do nome do branch*). Então qualquer commit que criássemos, apenas continuaria a construir nosso *main*.

Veja, se déssemos apenas um *commit* para criar C3, teríamos isso:

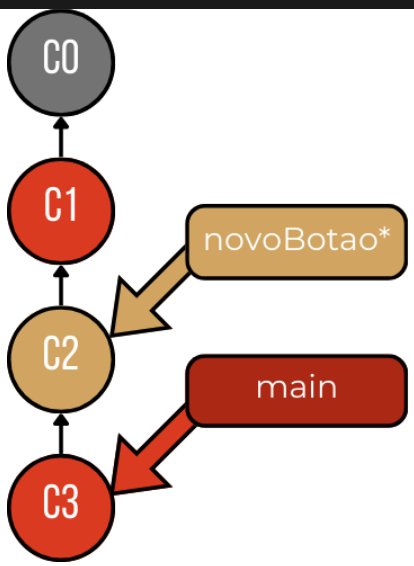


Como nosso objetivo é dar o *commit* no novo ramo, precisamos mudar a nossa posição para ele. Isso pode ser através do comando checkout. ele é útil para navegar através de diferentes ramos na árvore. O comando é

```
git checkout <nome-do-ramo>
```

No nosso exemplo, usaríamos:

```
git checkout novoBotao
```



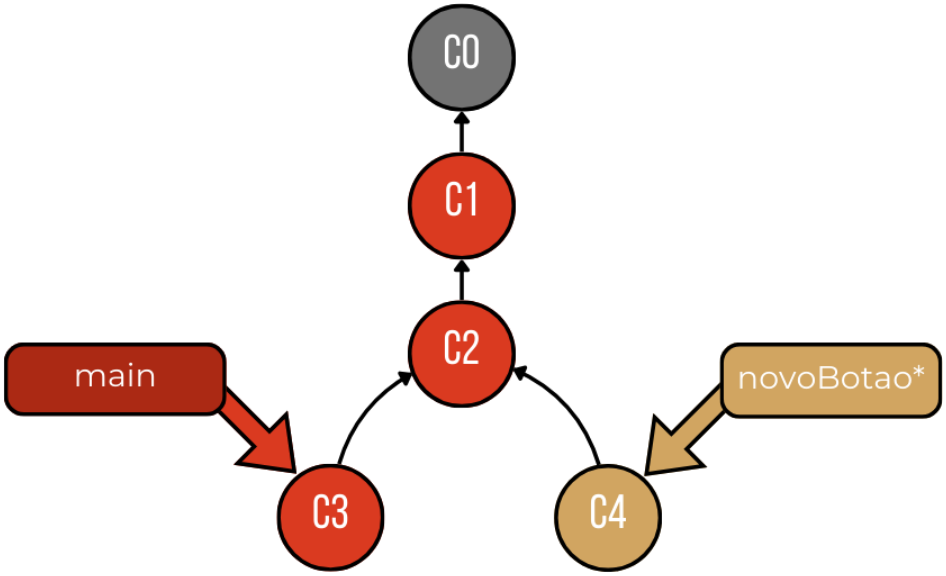
Veja que agora estamos com o asterisco em *novoBotao*, indicando que tudo que fizermos a partir de agora acontecerá na ramificação que criamos.

Poderíamos, também, ao invés de fazermos 2 comandos (criar o *branch* e mudar para ele), congregarmos isso num comando só. Para isso, adicionamos uma diretiva de *build*, colocando `-b` no comando de *checkout*. Assim, iremos sair do *branch* atual e construir o *branch* de destino. A sintaxe genérica do comando é:

```
git checkout -b <nome-do-ramo>
```

Vamos dar mais um *commit*, criando o C4. Agora, como estamos já no *branch* novoBotao, ele será criado nessa ramificação.

```
git commit -m "Commit C4"
```



E pronto: criamos o novo ramo e estamos trabalhando nele. Lembre sempre que o asterisco ao lado do nome do *branch* indica em qual deles estamos atualmente.

(CEBRASPE/TRE TO/2017) Considerando um programa em linguagem Java, assinale a opção que apresenta o comando do versionador Git que permite criar uma *branch* de nome *new_branch* e mudar para essa *branch* ao mesmo tempo.

a) *git log new_branch*
b) git clone new_branch
c) git checkout -b new_branch
d) git init new_branch
e) git commit -m 'new_branch'

Comentários:

O comando para que possamos criar e ir para o *branch* ao mesmo tempo é o *git checkout -b* – no caso da questão, *git checkout -b new_branch*. *(Gabarito: Letra C)*

Unindo Branches

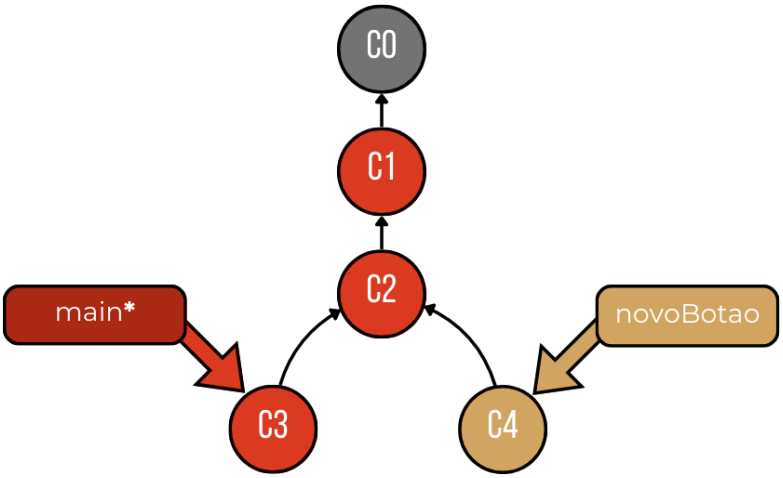
Merge

Terminamos todas as implementações que queríamos no *novoBotao*. E agora precisamos adicionar essas mudanças no repositório central – no *main*. Para fazermos isso, usamos o comando merge. Sua sintaxe é:

```
git merge <branch>
```

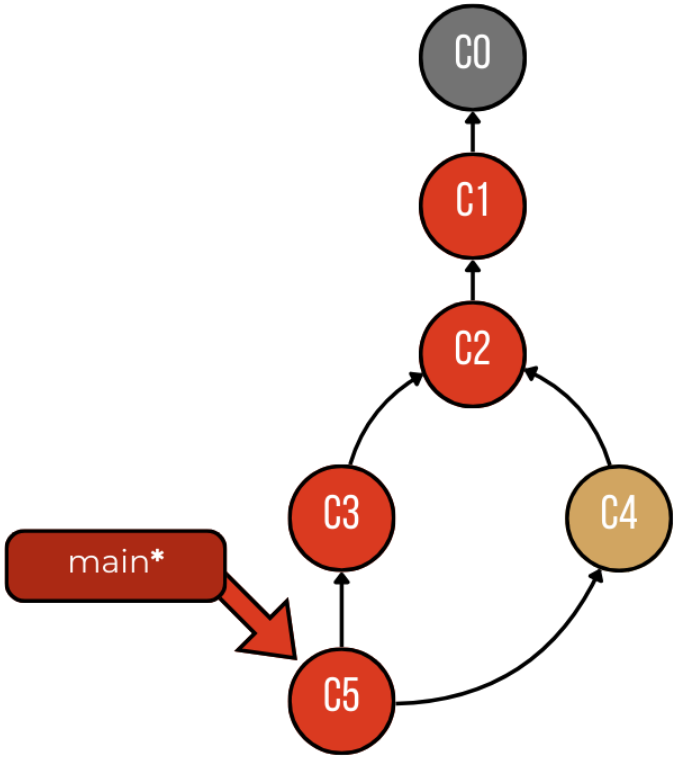
Mas ATENÇÃO! O *merge* irá juntar alterações feitas no *branch* que delimitarmos dentro do *branch* que estamos atualmente. Então, no nosso exemplo, antes de juntarmos as mudanças no *main*, precisamos selecioná-lo. Então vamos usar rapidamente o *checkout*, para mudarmos do *novoBotao* para o *main*. Preste atenção na mudança do ponteiro (asterisco).

```
git checkout main
```



Agora sim, podemos aplicar o *merge* e juntar a nova funcionalidade, *novoBotao*, ao ramo principal.

```
git merge novoBotao #vai criar o C5
```



02648216103 - HENRIQUE AMORIM BARRETO

Como C3 e C4 compartilham uma ancestralidade, ou seja, uma estrutura-base comum, a junção dos dois em C5 ocorreu sem problema – trouxemos a base de C2, as mudanças de C3 e de C4, e criamos o C5.

Perceba que conseguimos começar a visualizar o ramo principal, o "tronco" da árvore, em vermelho, e suas ramificações surgindo. Com isso, podemos voltar a qualquer ponto no tempo para gerenciar nossa aplicação.

Rebase

Rebase é uma segunda forma que temos de juntar os *branches* de uma árvore. Ele é usado para integrar as mudanças de diferentes *branches* de uma forma mais limpa e linear do que o *merge*. Em termos simples, o ***rebase* irá reescrever o histórico de *commits* da *branch* que está sofrendo esse *rebase***. Ele recalcula cada *commit* feito a partir do momento em que tivemos a ramificação (no nosso caso, C2).

A primeira grande diferença entre *rebase* e *merge* é a forma como eles unem os *branches*:

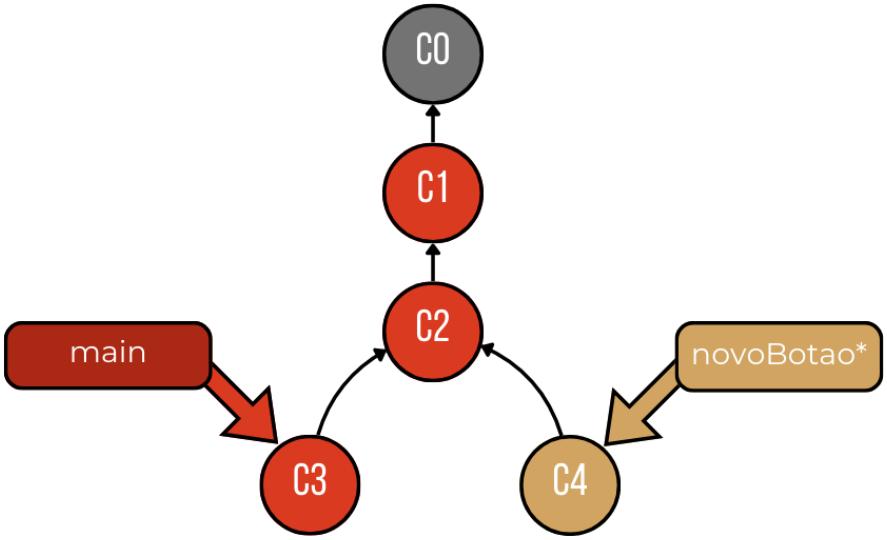
- O ***merge*** cria um novo commit combinando alterações de ambos os ramos
- O ***rebase*** reescreve o hsitórico de commits de forma linear, como se tivessem feitos em série, e não em paralelo

A sintaxe do comando é a seguinte:

```
git rebase <nome-da-branch>
```

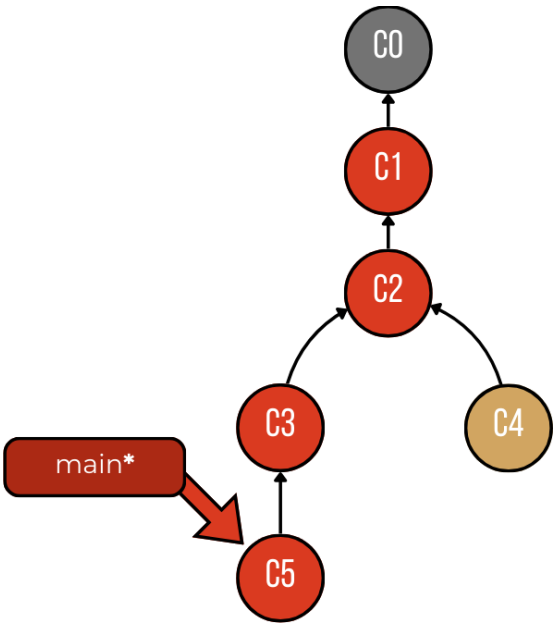
Aqui reside nossa segunda diferença. No *merge*, estávamos lá na *main* e "puxamos" o *novoBotao*. Aqui é diferente, precisamos estar dentro do *branch* que queremos que se una a outro. Então, executaremos o *rebase* de dentro do *novoBotao*, e definiremos em <nome-do-branch> para onde queremos que ele vá.

No nosso exemplo, vamos voltar 2 passos, estamos aqui:



Estamos na *novoBotao*, e queremos dar o *rebase*, uni-la ao *main*. Dessa forma, para criar o C5, usaremos o *rebase*:

```
git rebase main
```



Veja que temos o C5, novamente, com todas as mudanças feitas no C4 implementadas nele. Porém, visualmente, parece que o fluxo do C3 para o C5 foi em sequência quando, na verdade, se deu em paralelo. Dessa forma conseguimos deixar o fluxo de trabalho e controle de versões mais "redondinho".

Posicionando-se

Você pode ter percebido que é um pouco difícil se situar nos *branches*, ainda mais quando estamos com projetos enormes. A posição em que estamos no momento, ou seja, o último *commit* que recebeu o *checkout* recebe um nome simbólico – a "cabeça", ou HEAD.

O HEAD **sempre** apontará para o *commit* mais recente do ramo que estivermos trabalhando. Por isso, sempre que dou um *checkout* para determinado *branch*, estamos pulando para o *commit* mais recente.



(VUNESP/UNICAMP/2023) O Git possui um ponteiro especial utilizado para indicar o branch atual que está sendo trabalhado. O nome deste ponteiro é:

a) WORKPLACE
b) MASTER
c) SOURCE
d) TREE
e) HEAD

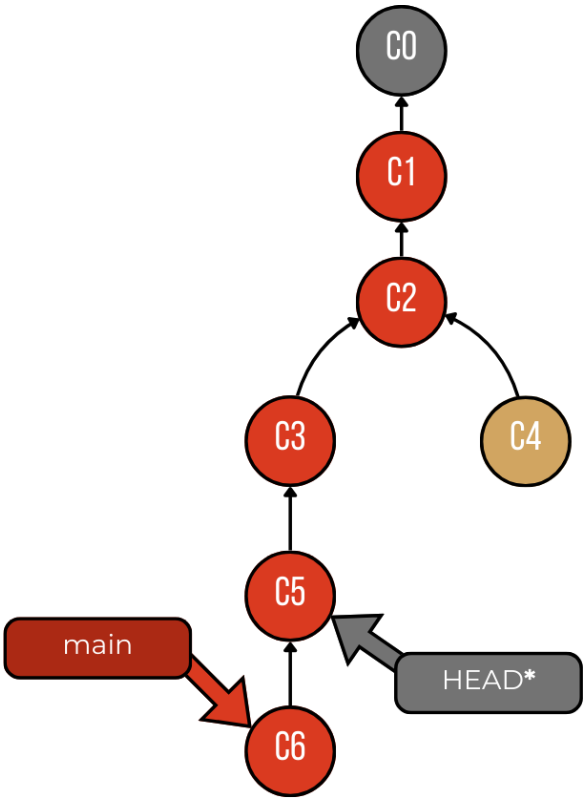
Comentários:

Para indicar a posição atual em que estamos trabalhando no *branch*, ou seja, para apontar ao *commit* atual que está sendo trabalhado, o Git usa um *alias* chamado de HEAD. Esse HEAD não necessariamente é o *commit* mais recente, podendo estar em qualquer posição do fluxo de trabalho. *(Gabarito: Letra E)*

Nós podemos soltar a cabeça, soltar o HEAD, anexando-o a um *commit* diferente do mais recente. Isso é simples – para fixar o HEAD em determinado *commit* realizado anteriormente, basta darmos o *checkout* pelo *hash* de cada *commit*. No nosso caso, o *hash* corresponde ao símbolo que colocamos dentro do círculo (C1, C2, C3, etc).

Vamos aumentar nosso *main* e, posteriormente, deixar o *head* fixado em C5, por exemplo.

```
git commit -m “Commit C6” #cria o C6
 git checkout C5 #fixa o HEAD no C5
```



E pronto, veja que navegamos para um *commit* anterior do mais recente.

Só que na vida real não temos essa árvore bonitinha que eu estou fazendo para entendermos o que está acontecendo. Para isso, usamos o comando git log. Ali, poderemos ver os *hashes*, que são gerados a partir de algum algoritmo, como o SHA-1.

Para facilitar a prática do dia a dia, o Git oferece algumas formas de posicionamento relativo. As duas principais formas são:

- `^` - Faz referência ao *commit* pai. Por exemplo, `HEAD^` refere-se ao *commit* anterior ao atual (onde o HEAD está apontando)
- `~n` - Referência ao n-ésimo commit ancestral. Por exemplo, `HEAD~3` refere-se ao terceiro *commit* anterior ao atual

Além disso, podemos forçar o ponteiro de determinado *branch* a se mover para outro lugar. No nosso caso, poderíamos mudar o *main* de C6 para C4. Para isso, usando o comando *branch*, adicionamos a diretiva `-f`. Veja:

```
git branch -f main C4
```

Essa sintaxe levaria nosso ponteiro do *main* lá pro *comit* C4.

Revertendo Mudanças

Para revertermos mudanças, temos duas abordagens diferentes: **reset** e **revert**.

Reset

O **reset** envolve mudar o ponteiro HEAD para determinada localização e remover todos os *commits* feitos após ele. Assim, podemos desfazer tanto mudanças locais, quanto *commits* completos.

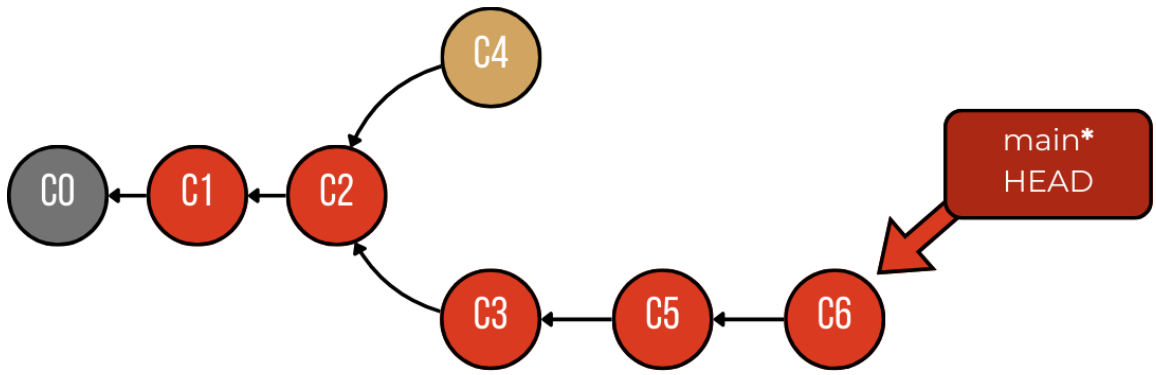
A sintaxe base é:

```
git reset [modo] [posição]
```

Temos três modos diferentes:

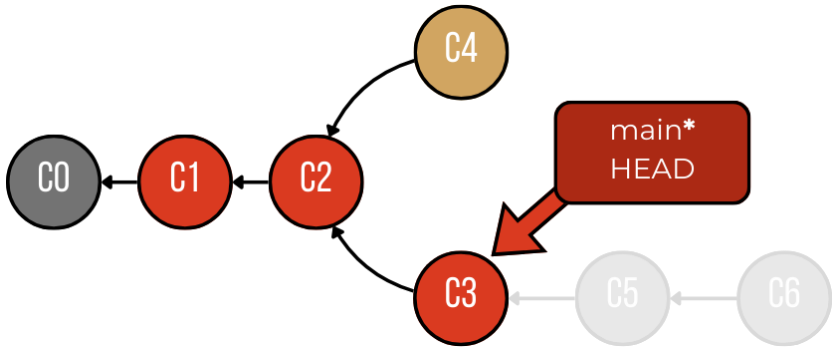
- `--soft` : Move HEAD para o commit especificado, mantendo as alterações no índice (staging area) e no diretório de trabalho. Os arquivos alterados serão listados como "Changes to be committed" (alterações a serem commitadas).
- `--mixed` : Move HEAD para o commit especificado, desfazendo o commit, mas mantendo as alterações no diretório de trabalho. As alterações serão listadas como "Changes not staged for commit" (alterações não preparadas para commit).
- `--hard` : Move HEAD para o commit especificado e descarta todas as alterações no índice e no diretório de trabalho desde então. As alterações são perdidas.

Digamos que a gente tenha a seguinte situação:



Acontece que os *comits* C5 e C6 foram errados, e precisamos voltar para o C3, apagando tudo com o *reset*. Usaremos a seguinte sintaxe:

```
git reset --hard HEAD~2
```

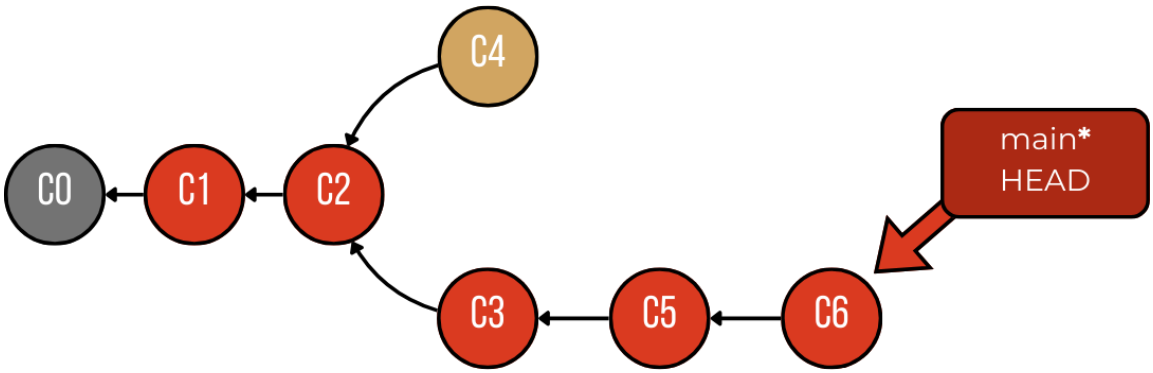


Os *commits* C5 e C6, que aparecem em cinza quase transparente acima, foram deletados, e retornamos para o *commit* C3.

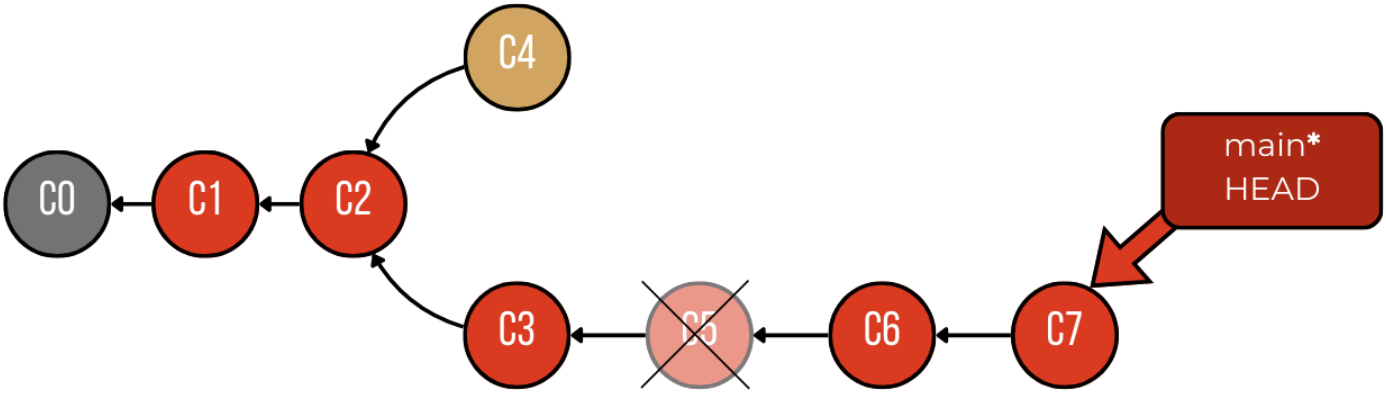
Revert

O revert é uma abordagem muito mais "leve". Ele cria um novo *commit* que desfaz as alterações de determinado *commit*. Ele é muito útil quando determinado *commit* no histórico apresenta problema, ou quando queremos manter um histórico mais linear e intacto.

Vamos voltar à situação anterior:



Digamos que o C5 possui um problema que está afetando a funcionalidade do programa. Vamos reverter apenas ele, criando um C7 novo, em que as mudanças do C5 deixam de existir.



Veja que a mudança entre os comandos é muito grande.

E bom, você aprendeu o básico sobre Git – que irá movimentar 90% do que ele oferece. Agora, iremos aprender uns comandos mais avançados e específicos que costumam a cair em prova.

(FGV/SEF MG/2023) O comando git que deve ser usado para descartar commits que foram feitos apenas localmente e traz o repositório local para o estado do último push do repositório remoto é o

a) git reset.
b) git revert.
c) git clean.
d) git stash.
e) git discard.

Comentários:

Quando queremos destacar commits locais e retornar a um estado anterior, o comando aplicado é o *git reset*. *(Gabarito: Letra A)*

Comandos Avançados

Cherry Pick

O cherry-pick é usado para pegar um *commit* específico e aplicá-lo em uma outra *branch* diferente. Podemos, com isso, mover um *commit* de uma *branch* para outra, fazendo uma movimentação dinâmica.

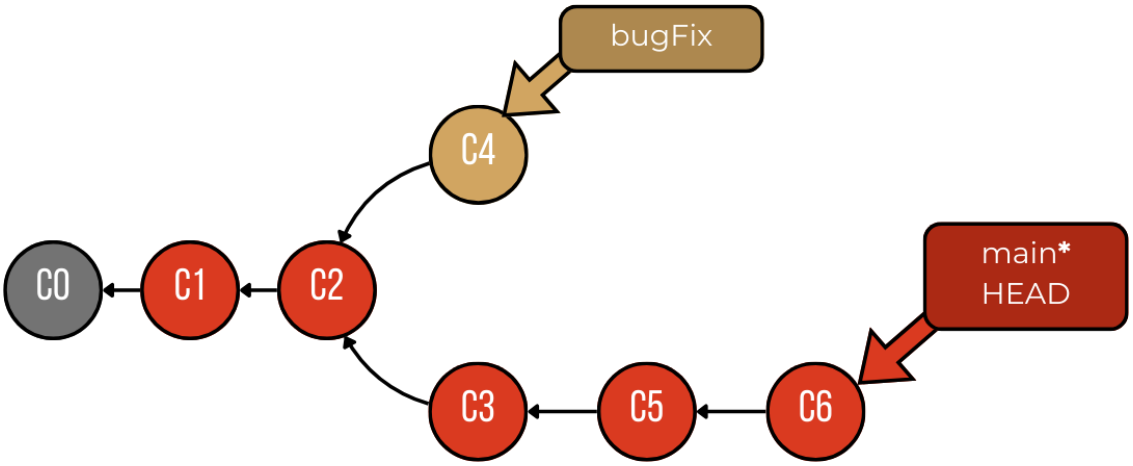
Isso é útil, por exemplo, quando criamos uma *branch* nova para correção de bug. Após implementarmos todas as necessidades, podemos jogar essa correção para dentro do *branch* principal, incorporando-o a eventuais mudanças que ocorreram no *main*.

A sintaxe do comando é:

```
git cherry-pick <commit-hash>
```

Aqui, temos de estar na *branch* que queremos adicionar o *commit*, e no <commit-hash> passamos o *hash* do *commit* que queremos que se incorpore à *branch* atual.

Por exemplo, imagine a situação abaixo:



Podemos implementar C4 no commit C6. Para isso, basta garantirmos que ponteiro está apontando para o *branch* que queremos implementar e usar o comando:

```
git cherry-pick C4
```

Com isso, criaremos o um novo *commit* dentro do *branch* onde implementamos o comando.



HORA DE PRATICAR!

(FUNDATEC/IFC/2023) Em relação ao comando "cherry-pick" da ferramenta de versionamento GIT, analise as assertivas abaixo, assinalando V, se verdadeiras, ou F, se falsas:

( ) Realiza o clone de um repositório.
( ) Permite selecionar commits específicos para trazer a sua branch desejada.
( ) Pode causar commits duplicados.
( ) Libera a edição do hash de um commit.
A ordem correta de preenchimento dos parênteses, de cima para baixo, é:

a) V – V – F – V.
b) F – V – V – F.
c) F – V – F – F.
d) F – V – V – V.
e) V – F – V – F.

Comentários:

Vamos lá, analisar cada afirmativa.

( ) Realiza o clone de um repositório.
FALSO. Não temos essa funcionalidade com o cherry-pick. O clone é feito a partir do comando *git clone*.
( ) Permite selecionar commits específicos para trazer a sua branch desejada.
VERDADEIRO. Podemos selecionar um ou mais *commits* específicos e trazê-los para a *branch* que desejarmos.
( ) Pode causar commits duplicados.
VERDADEIRO. Esse é um dos pontos negativos do cherry-pick, que podem aumentar o tamanho do repositório.
( ) Libera a edição do hash de um commit.
FALSO. O *hash* é o identificador único de um *commit* – não temos como alterá-lo a partir do *cherry-pick*.

Portanto, temos F – V – V – F. *(Gabarito: Letra B)*

02648216103 - HENRIQUE AMORIM BARRETO

Trabalhando com Repositórios Remotos

Até agora, todos os comandos que vimos trabalham com um repositório local, na própria máquina. Acontece que, na maior parte dos casos, estamos trabalhando com repositórios centrais remotos. Para lidar com isso, veremos três comandos: fetch, push e pull.

Fetch

O `git fetch` é usado para baixar os *commits*, as *branches* e demais objetos de um repositório remoto, para o repositório local. Os arquivos baixados não serão mesclados em nada no repositório local – é como se baixássemos um arquivo cru.

Para executar o comando é simples:

```
git fetch [opções] <nome-do-repositório> <ref-spec>
```

Dentre as opções, a principal opção é o `--prune` ou `-p`. Ele remove as referências locais que não existem no repositório remoto.

A `<ref-spec>` refere-se a uma referência específica, caso você queira baixar determinado elemento dentro do *fetch*, como um *branch* ou um *commit* específico.

Pull

O `git pull` busca as atualizações do repositório remoto, comparando-as com o repositório local, e as mescla automaticamente com as *branches* locais. Ele, basicamente, combina o *git fetch* e o *git merge* num comando só.

Sua sintaxe é:

```
git pull [opções] <nome-do-repositório> <ref-spec>
```

Podemos, como opção, além do *prune* que vimos no *fetch*, adicionar `-r` ou `--rebase`, assim, ao invés de fazermos um *merge*, fazemos um *rebase*, aplicando os *commits* em cima do *branch* atual.

Push

Diferentemente dos comandos anteriores, o `git push` é usado para enviar as alterações locais para um repositório remoto. Sua sintaxe é:

```
git push <nome-do-repositório>
```

Podemos definir o repositório remoto como padrão para os *commits*, sem necessidade de usar *push* toda vez. Isso é feito definindo uma *upstream*, adicionando a diretiva `-u` ou `--set-upstream` no comando do *push*.



CUIDADO!

Já falamos sobre esse comando no começo da aula, mas cuidado! O comando `git clone` não é um comando usado para sincronizar com repositórios remotos. O clone faz um download do repositório remoto e o recria de forma local. O repositório passa a ser local, não é mais remoto.

(COMPERVE/UFRN/2023) O Git foi construído para ser uma ferramenta de controle de versão distribuída e permite compartilhar projetos em repositórios remotos. Após o commit local de alterações realizadas em um projeto compartilhado, a fim de enviar essas alterações para a branch master do repositório remoto, deve-se utilizar o comando

a) git push origin master.
b) git clone origin master.
c) git pull origin master.
d) git purge origin master.

Comentários:

Para enviarmos alterações do *branch* local para o remoto, usamos o comando *git push*. *(Gabarito: Letra A)*

Monitoramento

Temos dois comandos principais para monitorar nosso versionamento: status e log.

Status

O comando **`git status`** é usado para exibir informações sobre o repositório local. O comando mostra quais arquivos foram modificados, quais arquivos estão na *staging area*, e também indica

se há alguma alteração no repositório remoto que ainda não foi implementada no repositório local. Sua sintaxe é simples:

```
git status
```

Log

O `git log` é um comando destinado a mostrar o histórico de *commits* do repositório. Ele mostra informações detalhadas sobre cada *commit*, passando informações como o autor, o *hash*, data, hora, mensagem, entre outros. Sua sintaxe é:

```
git log [opções]
```

Dentre as opções, podemos passar os argumentos:

- `--oneline` → mostra as informações de cada commit em uma única linha
- `--graph` → exibe o histórico de *commits* em forma de grafo
- `--author=" "` → exibe os *commits* de determinado autor
- `--since=" "` → exibe os *commits* a partir de uma data especificada
- `--until=" "` → mostra os *commits* até uma data especificada
- `--follow=" "` → foca em um arquivo específico



(FGV/TRT 16/2022) O comando que pode ser executado na ferramenta de versionamento Git para exibir a lista dos arquivos alterados no diretório de trabalho local é

a) git fetch
b) git gc
c) git status
d) git prune
e) git remote

Comentários:

Questão tranquila – para exibirmos a lista de arquivos alterados no repositório local, precisamos verificar o estado deles. Dessa forma, rodamos o comando git status. *(Gabarito: Letra C)*

Bisect

O `git bisect` é um comando focado em encontrar *commits* específicos que tenham introduzido determinado bug no projeto. Ele usa uma busca binária para localizar o *commit* responsável, reduzindo – e muito – o número de verificações necessárias para localizar o erro.

Como é um comando interativo, ele funciona de uma forma um pouco diferente:

- Iniciamos o bisect com o comando `git bisect start` ;
- Apontamos para um *commit* ruim, com bugs, com `git bisect bad <nome>` ;
- Apontamos para um *commit* bom, sem bugs, com `git bisect good <nome>` ;
- O Git apresentará resultados, e você informa a ele se o *commit* é o responsável por introduzir o *bug*
- Finalizamos o comando com `git bisect reset` ;

Um exmplo:

```
git bisect start
git bisect bad commit_bugado_conhecido
git bisect good commit_bom_conhecido
```

(FGV/CÂMARA DOS DEPUTADOS/2023) Marcos trabalha em uma equipe de desenvolvimento de software que utiliza o Git como plataforma de gestão de configuração. Um membro da equipe, no entanto, fez um commit que introduziu, de maneira não intencional, um código com comportamento errático.

Assinale a opção que indica o comando do Git que Marcos utilizou para fazer uma pesquisa binária para descobrir qual commit, no histórico do seu projeto, introduziu esse problema.

a) bisect.
b) fetch.
c) find.
d) locate.
e) rebase.

Comentários:

O comando usado para fazer uma busca binária e encontrar um *bug* nos commits é chamado de *git bisect*. *(Gabarito: Letra A)*

Grep

Assim como em várias outras ferramentas de CLI, o **`git grep`** funciona pesquisando algum arquivo específico no repositório Git. Ele funciona de forma similar ao *grep* do Unix, mas adaptado para o ambiente do Git.

A sintaxe básica é simples:

```
git grep [opções] <padrão>
```

Dentro as opções, podemos definir:

- `-i` ou `--ignore-case` → ignora diferença entre maiúsculas e minúsculas
- `-w` ou `--word-regexp` → procura apenas por palavras inteiras que correspondam ao padrão
- `-e` ou `--extended-regexp` → permite usar expressões regulares estendidas
- `--all-match` → exibe apenas arquivos que correspondam a todos os padrões

Digamos que foi incluído um arquivo que trocou a palavra "length" por "lenght" (um erro bem comum, diga-se de passagem rs). Podemos pesquisar a palavra errada com o *grep* da seguinte forma:

```
git grep -w lenght
```

Stash

Usamos o comando **`git stash`** para armazenar temporariamente as alterações locais que ainda não foram *commitadas*, permitindo que você as remova da sua área de trabalho sem perder alterações. Isso é útil quando você precisa alternar entre tarefas e não pode ficar esperando o *commit*.

A sintaxe base é:

```
git stash [opções]
```

Dentre as opções, temos:

- `list` → lista todas as *stashes* armazenadas

- `apply` → aplica a última stash armazenada
- `pop` → aplica a última stash armazenada e a remova da lista de stashes
- `drop` → remove uma stash específica da lista
- `clear` → limpa todas as stashes armazenadas

(CEBRASPE/SERPRO/2021) Quanto ao uso de Git para versionamento, julgue o item a seguir.

A execução do comando git stash sem argumentos por padrão é similar à execução do comando git stash show, na medida em que ambas mostram as alterações armazenadas por este comando.

Comentários:

Não... se passarmos o comando *stash* sem argumentos, colocaremos o *commit* atual em que estamos trabalhando na lista. Se usarmos *show*, temos o mesmo efeito do *list*, que irá mostrar uma lista com todos os *stashes* criados. *(Gabarito: Errado)*

(FGV/TJ RN/2023) A desenvolvedora Amélia está trabalhando no projeto AppBravo. O controle de versões do código-fonte do AppBravo é feito pelo Git. Logo após realizar algumas modificações no branch um do código-fonte do AppBravo, Amélia precisou alterar a cópia de trabalho para o branch dois. Para não perder o progresso feito no branch um, que não estava pronto para ser commitado, Amélia utilizou o comando do Git que arquiva localmente as alterações realizadas que não foram commitadas.

Portanto, Amélia utilizou o comando do Git:

a) diff;
b) fetch;
c) stash;
d) reset;
e) rebase.

Comentários:

Quando precisamos criar um espaço "intermediário", para armazenarmos arquivos que não estão prontos para serem commitados, que não queremos perder, porque precisamos limpar nossa área para trabalhar em outra versão, branch, ou o que seja, usamos o *stash*, que irá armazenar esses arquivos até que tomemos alguma decisão sobre o que fazer com eles. (Gabarito: Letra C)

Diff

O comando `git diff` é usado para mostrar as diferenças entre o estado atual do repositório, e um estado anterior. Ele é excelente para realizarmos comparações entre *commits* ou *branches* diferentes.

Sua sintaxe base é:

```
git diff [opções] <commit1> <commit2>
```

Como opções, podemos adicionar:

- `--cached` ou `--staged` → mostra as diferenças entre a staging area e o último commit
- `--name-only` → mostra apenas os nomes dos arquivos modificados, sem mostrar as diferenças
- `--stat` → mostra um resumo estatístico das alterações

Por exemplo, podemos comparar um *commit* específico com o HEAD.

```
git diff HEAD commit_generico
```

Demais Comandos

Como o Git possui uma penca de comandos, e já vimos os principais, vou listar numa tabelinha todos os comandos que eu me lembrar – inclusive alguns que já vimos aqui na aula. Use essa tabela como uma forma de revisão e de aprofundamento em comandos que não vimos durante a aula.

COMANDO	EXPLICAÇÃO	EXEMPLO
`tag`	Marca um commit com uma versão	git tag v1.0
`config`	Configura variáveis do Git	git config --global user.name "Seu Nome"
`show`	Mostra informações sobre um objeto	git show HEAD
`blame`	Mostra quem modificou cada linha	git blame arquivo.txt
`reflog`	Mostra o histórico de referências	git reflog
`submodule`	Gerencia submódulos	git submodule add <url>
`archive`	Cria um arquivo a partir de um commit	git archive --format=zip HEAD -o arquivo.zip
`remote`	Gerencia repositórios remotos	git remote add origin <url>
`cherry-pick`	Aplica um commit em outra branch	git cherry-pick <commit>

rebase	Reaplica commits em outra base	git rebase main
reset	Reseta o estado do repositório	git reset --hard HEAD^
revert	Desfaz um commit	git revert <commit>
branch	Gerencia branches	git branch -d <branch>
checkout	Alterna entre branches ou revisões	git checkout <branch>
merge	Mescla duas ou mais branches	git merge <branch>
fetch	Busca alterações do repositório remoto	git fetch origin
pull	Busca e mescla alterações do repositório remoto	git pull origin main
push	Envia commits para um repositório remoto	git push origin main
status	Mostra o estado do repositório	git status
log	Mostra o histórico de commits	git log
diff	Mostra as diferenças entre commits	git diff HEAD^ HEAD
stash	Armazena temporariamente alterações	git stash save "mensagem"
clean	Remove arquivos não rastreados	git clean -df
grep	Pesquisa em arquivos rastreados	git grep "palavra"
init	Inicializa um repositório Git	git init
clone	Clona um repositório Git	git clone <url>
add	Adiciona arquivos à staging area	git add arquivo.txt
commit	Registra as alterações no repositório	git commit -m "Mensagem"
rm	Remove arquivos do repositório	git rm arquivo.txt
mv	Move ou renomeia arquivos	git mv arquivo.txt novo_nome.txt
ls-files	Lista arquivos rastreados	git ls-files
show-branch	Mostra branches e seus commits	git show-branch
merge-base	Encontra o commit base de dois commits	git merge-base <commit1> <commit2>
cat-file	Mostra conteúdo de objetos	git cat-file -p <hash>
rev-parse	Converte nomes em hashes	git rev-parse HEAD
bisect	Ajuda a encontrar um commit problemático	git bisect start
blame	Mostra quem modificou cada linha	git blame arquivo.txt
remote	Gerencia repositórios remotos	git remote add origin <url>
mergetool	Abre uma ferramenta de merge	git mergetool



`instaweb`	Inicia um servidor web para visualização	git instaweb
`help`	Exibe ajuda sobre comandos Git	git help commit
`version`	Exibe a versão do Git	git version

(FGV/TJ SE/2023) Carlos está resolvendo um conflito de merge em um projeto que utiliza a ferramenta de versionamento Git. A fim de elucidar o conflito, Carlos recorreu ao comando git capaz de exibir cada linha de um determinado arquivo junto a informações do último commit que modificou a linha, incluindo o autor do commit. Carlos recorreu ao comando git:

a) log;
b) tag;
c) diff;
d) stash;
e) blame.

Comentários:

O comando responsável por mostrar quem modificou cada linha de um código, usado para apontar o "culpado", digamos assim, é o *git blame*. *(Gabarito: Letra E)*

(IDECAN/SEFAZ RR/2022) Assinale a alternativa que contém o comando git responsável por encontrar um commit que está gerando um bug ou uma inconsistência entre uma sequência de commits.

a) bisect
b) rebase
c) stash
d) switch
e) grep

Comentários:

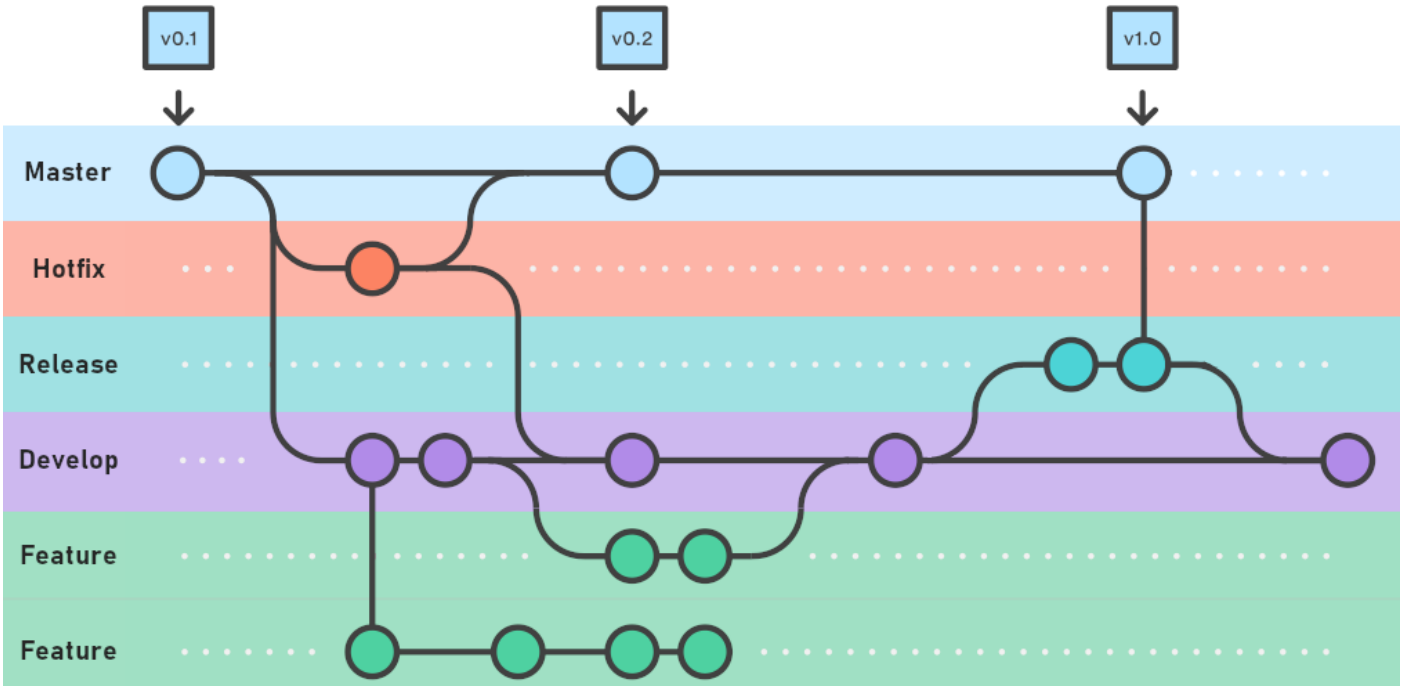
O comando que faz uma busca binária por determinado bug ou incosistência, tendo *commits* bons e ruins como referência, é o *git bisect*. *(Gabarito: Letra A)*

Extensões

Git Flow

Git Flow é um modelo alternativo de ramificações pra o Git, criado em 2010. Ele define um padrão para organizarmos os *branches* em um repositório, com o objetivo de facilitar o desenvolvimento colaborativo e tornar a linguagem utilizada nas ramificações mais universal.

A ideia por trás do GitFlow é classificar as ramificações em 5 tipologias diferentes.



- Master: A branch master é considerada a branch principal do projeto. Ela contém o código fonte que reflete a produção atual, ou seja, o código que está em produção.
- Develop: A branch develop é usada como ramificação de integração para recursos. É a branch para onde as novas funcionalidades são mescladas e testadas antes de serem incorporadas à branch master.
- Feature: Cada nova funcionalidade é desenvolvida em uma branch de feature separada, que é criada a partir da branch develop. Uma vez concluída, a branch de feature é mesclada de volta para develop.
- Release: Quando a develop atinge um estado estável e está pronta para ser implantada, uma branch de release é criada a partir da develop. Qualquer ajuste final necessário pode ser feito nesta branch antes da sua mesclagem na master para implantação.

- **Hotfix:** Se um problema crítico é descoberto na produção que precisa ser corrigido imediatamente, uma branch de hotfix é criada a partir da master, o problema é corrigido nesta branch e então é mesclada tanto na master quanto na develop.

Por exemplo – digamos que tenhamos uma funcionalidade para ser desenvolvida. Para isso, precisamos criar uma *branch feature*. Usamos o código abaixo:

```
git flow feature start <nome_da_feature>
```

(AOCP/CM TERESINA/2021) Jaqueline é uma analista que planeja utilizar Git e Gitflow em seu projeto. Assinale a alternativa que apresenta uma prática inconsistente com esse processo.

a) Realizar um commit na branch develop.
b) Criar um Pull Request para a branch develop.
c) Criar uma feature branch.
d) Criar uma hotfix branch.
e) Criar uma release branch.

Comentários:

Das alternativas apresentadas, a única que apresenta uma inconsistência é a letra A. Isso porque os *commits* são feitos na *branch release* e *main*. *(Gabarito: Letra A)*

(FGV/SEN/2022) O gitflow é recomendado para projetos que utilizam versionamento semântico.

Assim sendo, ele propõe o uso dos branches master e develop para linha contínua de projeto, além de branches de suporte, que são do tipo

a) feature, hotfix ou release.
b) feature, test ou release.
c) implement, fix ou release.
d) hotfix, fix ou release.
e) release, package ou test.

Comentários:

O GitFlow propõe, além das *branches* master e develop, 3 branches de suporte – feature, hotfix e release. Portanto, correta a letra A. *(Gabarito: Letra A)*

Trunk-based development

Trunk-based Development (TBD) é uma estratégia de ramificação em que todos os desenvolvedores trabalham em uma única branch principal (chamada de trunk, main ou master).

A ideia central é simples: em vez de criar branches de longa duração como no Git Flow, os desenvolvedores fazem commits frequentes e pequenos diretamente na branch principal — ou criam branches de curtíssima duração (geralmente com vida útil de poucas horas a, no máximo, um ou dois dias).

"Mas professor, e se o código quebrar?" Excelente pergunta! Inclusive, para quem está acostumado com o Git Flow, a premissa do Trunk-based development parece ser um antipadrão de programação.

No TBD, a premissa é que o código na branch principal deve estar sempre em estado implantável (deployable). Para garantir isso, o modelo se apoia fortemente em práticas de integração contínua (CI) e entrega contínua (CD), com testes automatizados rodando a cada commit.

As principais características do Trunk-based development são:

Característica	Descrição
Branch principal única	Todo o trabalho converge para o **trunk** (main/master)
Branches de curta duração	Quando existem, duram horas ou poucos dias — nunca semanas
Commits pequenos e frequentes	Cada commit representa uma mudança incremental e testável
Integração contínua obrigatória	Testes automatizados validam cada integração
Feature flags	Funcionalidades incompletas são ocultadas por flags em vez de isoladas em branches

Olhe só: as feature flags (ou feature toggles) são um conceito-chave aqui. Em vez de manter uma funcionalidade em desenvolvimento isolada em uma branch separada (como a feature branch do Git Flow), o código é integrado na branch principal mesmo que incompleto, mas fica desativado por uma flag de configuração. Quando a funcionalidade estiver pronta, basta ativar a flag — sem precisar fazer merge de uma branch de longa duração.

Git Flow vs. Trunk-based development

É interessante contrastar o Trunk-based development com o Git Flow, pois eles são opostos em diversos aspectos: enquanto o Git Flow usa múltiplas branches de longa duração, o TBD concentra tudo na branch principal.

Aspecto	Git Flow	Trunk-Based Development
Número de branches	Múltiplas (master, develop, feature, release, hotfix)	Uma principal (trunk/main)
Duração das branches	Longa (semanas a meses)	Curtíssima (horas a dias)
Frequência de integração	Baixa (ao concluir features)	Alta (várias vezes ao dia)
Releases	Via branch release dedicada	Diretamente da branch principal
Complexidade	Alta: muitas branches e regras	Baixa: fluxo linear e direto
Ideal para	Projetos com releases planejadas e ciclos longos	Equipes ágeis com entrega contínua (CI/CD)

Empresas como Google, Facebook e Netflix adotam variações do Trunk-based development justamente por praticarem deploys contínuos, várias vezes ao dia. Já o Git Flow continua sendo popular em projetos que exigem ciclos de release mais controlados, como software embarcado ou projetos com múltiplas versões simultâneas em produção.

(PROF. VINICIUS BORGES / INÉDITA - 2026)
No modelo Trunk-Based Development, funcionalidades em desenvolvimento são isoladas em branches de feature de longa duração até que estejam completas e prontas para integração na branch principal.

Comentários:
No Trunk-Based Development, o isolamento de funcionalidades não se dá por branches de longa duração: essa é a abordagem do Git Flow. No TBD, funcionalidades incompletas são integradas diretamente na branch principal e controladas por feature flags (ou feature toggles), que permitem ativar ou desativar o recurso em produção. As branches, quando utilizadas, têm vida curtíssima (horas a poucos dias).

Gabarito: Errado

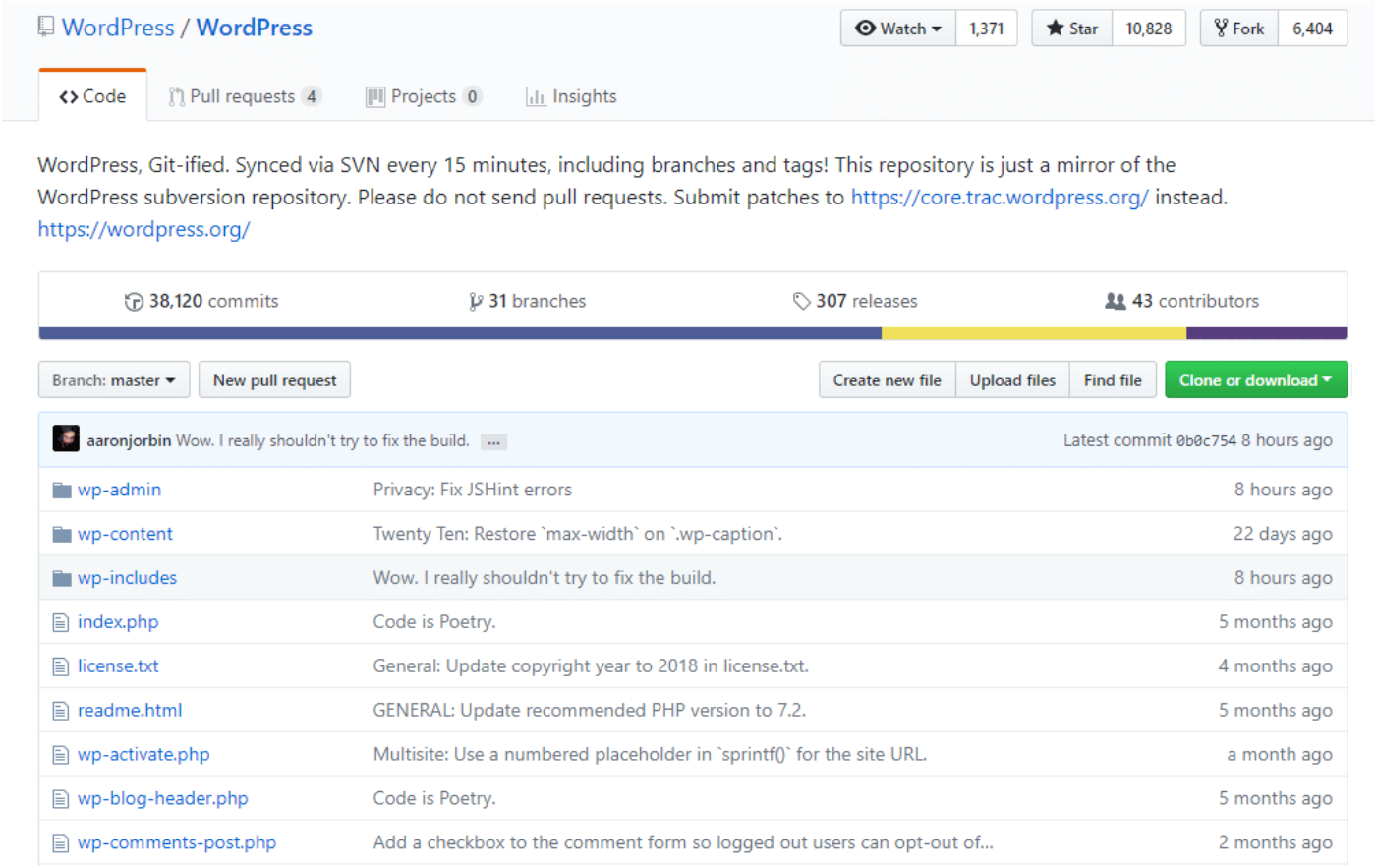
Git Hub

O GitHub é um site e serviço baseado na nuvem, que fornece um local para armazenamento centralizado de códigos, funcionando como um repositório centralizado para versionamento.

Entre desenvolvedores, o GitHub é quase como um requisito obrigatório para o mercado de trabalho, já que ele mostra a outros programadores o que determinado desenvolvedor está

fazendo – isso se ele opta por deixar o código da programação pública. Tornando-se, assim, o portfólio dos programadores.

O GitHub possui integração direta com o Git e ferramentas IDE, permitindo o carregamento de alterações em tempo real diretamente do ambiente de código. O repositório tem essa interface:



O GitHub fornece três funcionalidades muito úteis, adicionais ao Git:

- GitHub Actions – forma de facilitar a implementação de fluxos de trabalho, permitindo implantar *pipelines* CI/CD diretamente do Hub.
- GitHub Pages – fornece uma plataforma para hospedagem de sites. Digamos que você desenvolveu um código *web* e deseja hospedar um site diretamente a partir do seu repositório – o GitHub Pages permite isso. O Padrão para acesso de páginas é *https://nomeDoUsuario.github.io*.
- Fork – cria uma cópia independente de um repositório no Hub, fazendo com que qualquer alteração na cópia não afete o original.

Caso queira explorar mais o GitHub, acesse *https://github.com/* .



HORA DE PRATICAR!

(VUNESP/ISS SP/2023) O GitHub possui o recurso GitHub Pages, no qual é possível hospedar sites na plataforma. Desconsiderando domínios personalizados, uma página hospedada no GitHub por um usuário com nome user terá como link:

a) https://user.gh.io
b) https://user.gh.org
c) https://user.github.org
d) https://user.git
e) https://user.github.io

Comentários:

Como acabamos de ver, o site padrão para o Pages é https://user.github.io. Portanto, correta a letra E. *(Gabarito: Letra E)*

(CEBRASPE/ALECE/2021) No GitHub, um serviço web que oferece diversas funcionalidades extras aplicadas ao Git,

a) as pulls requests criam um repositório local no computador de modo a sincronizar o conteúdo desse local com o repositório do GitHub.
b) um branch é uma ação equivalente a um Commit no projeto, que marca a alteração no arquivo ou conjunto de arquivos por meio de um hash.
c) o blame é a capacidade de executar os serviços do GitHub em vários nós de modo que se tenha uma cópia do conteúdo contido no repositório em vários locais ao mesmo tempo.
d) pode-se criar fluxos de trabalho personalizados de integração contínua (continuous integration) e de implantação contínua (continuous deployment) diretamente no repositório do GitHub.
e) uma bifurcação (fork) é uma cópia de um repositório que atualiza automaticamente o repositório original a cada alteração realizada no projeto.

Comentários:

Vamos ver cada alternativa.

a) Errado. É o comando *git clone* que cria um repositório local sincronizado.

02648216103 - HENRIQUE AMORIM BARRETO

b) Errado. Como vimos na aula, *branches* e *commits* são coisas diferentes
c) Errado. O Blame é usado para encontrar quem fez determinada alteração no código
d) Certo. O responsável por fornecer um fluxo CI/CD é o GitHub Actions
e) Errado. O Fork é uma cópia que *não* atualiza o repositório original.

Portanto, correta a letra D. *(Gabarito: Letra D)*

QUESTÕES COMENTADAS

01. (CEBRASPE/CAU BR/2024) Quanto ao gerenciamento de configuração do *software* e aos serviços de mensageria, julgue o item a seguir.

Em um projeto de *software* que utilize a ferramenta Git para controle de versão, é recomendável que cada desenvolvedor trabalhe em sua própria *branch* local e faça *merge* com a *branch master* apenas quando o código estiver testado e revisado.

Comentários:

Essa, de fato, é uma boa prática e recomendada a todos os desenvolvedores. Dessa forma, quaisquer erros que ocorram afetarão apenas a aplicação localmente, sem "contaminar" o repositório central – se alinhando muito com as práticas de DevOps.

Gabarito: Certo

---

02. (CONSULPLAN/DPE PR/2024) O Git é um sistema de controle de revisão distribuído, rápido e escalável, com um conjunto de comandos que fornece operações de alto nível e acesso total aos componentes internos. Qual comando Git permite atualizar a versão local do repositório concurso a partir da versão remota?

a) git *checkout* concurso
b) git *pull origin* concurso
c) git *push origin* concurso
d) git *commit -m* "Atualização do repositório concurso"

Comentários:

Para atualizarmos o repositório local, precisamos "puxar" o conteúdo do repositório central – ou de um local de armazenamento, como o GitHub. Nessa caso, o comando a ser usado é o *git pull*.

Gabarito: Letra B

---

03. (CONSULPLAN/DPE PR/2024) O comando *git stash* do *Git* é empregado para acumular as alterações em um outro diretório, ou seja, quando quiser gravar a condição atual do diretório ativo e do índice; porém, querendo voltar para um diretório funcional limpo. O comando salva as suas alterações locais e reverte o diretório de trabalho para coincidir ao *commit HEAD*. Sobre o comando *git stash do Git*, assinale a afirmativa **INCORRETA**.

a) *git stash clear*: realiza a remoção em ordem de pilha.
b) *git stash apply*: restaura as modificações de um stash.
c) *git stash list*: lista as entradas acumuladas (stash) que existem.
d) git stash show: lista todos os arquivos modificados no ultimo stash.

02648216103 - HENRIQUE AMORIM BARRETO

Comentários:

Queremos a afirmativa *incorreta*. Vamos analisar cada uma.

a) Errado. git stash clear remove todos os stashes e não respeita uma "ordem de pilha".
b) Certo. git stash apply restaura modificações de um stash sem removê-lo.
c) Certo. git stash list exibe uma lista de todas as entradas do stash acumuladas.
d) Certo. git stash show lista os arquivos modificados no último stash, confirmando a descrição.

Portanto, incorreta a letra A.

Gabarito: Letra A

---

04. (CEBRASPE/ITAIPU/2024) Assinale a opção que apresenta o comando utilizado no Git para versionar o projeto com um pacote de alterações.

a) add
b) checkout
c) commit
d) clone
e) branch

Comentários:

Versionar um código significa adicionar uma nova versão do código à árvore. Isso é feito a partir do comando *git commit* – que confirma uma versão e a registra no repositório.

Gabarito: Letra C

---

05. (FGV/TJ AP/2024) A analista Cátia criou o branch tarefa10 em um repositório Git local. Logo após, Cátia alternou o branch de trabalho atual para o branch recém-criado, tarefa10. Estando no branch local tarefa10, Cátia precisou renomear o branch para t10_concluida, com um único comando da ferramenta git.

Para renomear o branch de trabalho atual com um único comando, Cátia executou no repositório Git local o comando do git:

a) mv t10_concluida;
b) reset t10_concluida;
c) rebase t10_concluida;
d) branch -m t10_concluida;
e) checkout -b t10_concluida.

Comentários:

O comando correto é o git branch -m t10_concluida. Ele é a maneira correta de renomear o branch atual no Git. Ele renomeia o branch sem alterar seu conteúdo ou histórico. Como o enunciado exige renomear o branch atual com um único comando, essa alternativa é a correta.

Gabarito: Letra D

---

06. (FGV/BBTS/2023) GIT é uma ferramenta utilizada para fazer controles de versões de projetos e seus arquivos.

Assinale a opção que apresenta os três possíveis estados em que os arquivos recém-criados ainda não foram submetidos a um snapshot.

a) commited, saved e shared.
b) commited, modified e staged.
c) consolidated, shared e logged.
d) consolidated, persisted e shared.
e) consolidated, confirmed e logged.

Comentários:

Temos três estados nos arquivos:

- Modified – arquivos foram modificados, e ainda não adicionados à staging area
- Staged – arquivos foram adicionados à staging area, por meio do comando *git add*
- Commited – alterações foram confirmadas e incorporadas ao repositório

Portanto, correta a letra B.

Gabarito: Letra B

---

07. (AOCP/IF MA/2023) Qual comando pode ser utilizado dentro do GIT para listar o histórico de versões para um arquivo, incluindo mudanças de nome?

a) $ git show [arquivo]
b) $ git log
c) $ git log --follow [arquivo]
d) $ git rm [arquivo]
e) $ git stash drop

Comentários:

Para acompanharmos um arquivo específico, usamos o comando *log* seguido da diretiva *--follow*. O comando ficaria *git log --follow [arquivo]*.

Gabarito: Letra C

---

08. (VUNSEP/TJ RS/2023) Git é um sistema de controle de versões utilizado no desenvolvimento de código. Caso um bug tenha aparecido em um sistema que está sendo desenvolvido, com o apoio do Git é possível identificar qual commit foi responsável por este bug, com o auxílio de um comando.

Este comando é o:

a) git-bisect
b) git-blame
c) git-grep
d) git-status
e) git-tag

Comentários:

Já deu pra perceber que esse comando cai bastante né? O comando responsável por detectar *bugs*, que usa uma busca binária, é o git bisect.

Gabarito: Letra A

---

09. (VUNESP/TJ RS/2023) Na ferramenta de versionamento Git, dentre as alternativas a seguir, o nome do arquivo que permite definir tipos de arquivo que, intencionalmente, não devem ser rastreados no controle de versão é:

a) .ignore
b) .gitignore
c) .gituntracked
d) gitfile
e) makefile

Comentários:

Quando queremos ignorar determinado arquivo ou *branch*, usamos o comando *git ignore*.

Gabarito: Letra B

---

10. (VUNESP/TJ RS/2023) Um dos sistemas de controle de versão mais populares para uso em desenvolvimento de software atualmente é o Git.

Com relação ao Git, assinale a alternativa que apresenta o comando que atualiza as referências locais com relações às remotas, informando ao Git local que ele deve obter as informações de metadados mais recentes do repositório original, mas não faz a integração e sobrescrita nos arquivos de trabalho locais.

a) status
b) pull
c) merge
d) fetch
e) remote

Comentários:

Quando queremos atualizar as referências locais, podemos usar dois comandos – o *pull*, que irá integrar e sobrescrever os arquivos, sincronizando-os; e o *fetch*, que não faz integração nem sobrescrita. Portanto, para essa questão, usaremos o *fetch*.

Gabarito: Letra D

---

11. (CEBRASPE/MPE RO/2023) Caso se pretenda criar, no desenvolvimento de um novo código em certo projeto de software, um espaço no repositório Git que seja independente do principal, a fim de fazer alterações sem interferências no código principal, então isso poderá ser feito por meio do uso do recurso denominado

a) branch.
b) commit.
c) release.
d) rollback.
e) restore.

Comentários:

A questão cobra conhecimentos acerca dos *branches*. Quando queremos fazer alterações sem interferirmos no código principal, criamos uma ramificação – um branch.

Gabarito: Letra A

---

12. (VUNESP/TJM SP/2023) Git é um sistema de controle de versões utilizado no desenvolvimento de soluções de software.

Um usuário que deseja copiar o conteúdo de um repositório público em sua máquina pode fazer isso por meio do seguinte comando:

a) git-clone
b) git-copy
c) git-init
d) git-add
e) git-push

Comentários:

Podemos trazer um repositório público na integra para o repositório local usando o comando de clone – *git clone*.

Gabarito: Letra A

---

13. (VUNESP/TJM SP/2023) Um usuário está manipulando arquivos em um repositório Git. Para adicionar todos os novos arquivos no Git, ele pode utilizar o comando

a) git add -a
b) git add -i
c) git add -n
d) git add –u
e) git add -v

Comentários:

Quando queremos adicionar todos os elementos, usamos a diretiva --all, ou -a. Quanto às demais:

- -i → inicia a interface interativa do Git, que permite escolher quais mudanças devem ser adicionadas à staging area
- -n ou --none → adiciona arquivos ao índice, mas não os atualiza com suas mudanças
- -u → adiciona apenas arquivos modificados e removidos à staging area, ignorando novos arquivos
- -v → adiciona arquivos à staging area e exibe informações verbosas sobre as operações realizadas

Gabarito: Letra A

---

14. (FCC/PGE AM/2022) Um Técnico utilizou corretamente um comando git para modificar a mensagem do commit mais recente, ou seja, o último commit feito por ele no projeto. Trata-se do comando git

a) add merge.
b) push.
c) commit --amend.
d) add message.
e) checkout master.

Comentários:

Quando queremos modificar algum *commit*, adicionando uma mensagem, por exemplo, usamos o *commit* seguido da diretiva de emenda - --amend. Portanto, o comando ficará *commit --amend.*

Gabarito: Letra C

---

15. (VUNESP/ALESP/2022) João criou uma nova ramificação (branch) no repositório Git chamada "fix-issue-35", submeteu o código por meio do comando commit e enviou as alterações para o repositório remoto por meio do comando push. Maria gostaria de contribuir com código nessa ramificação, então, para isso, ela precisa executar o(s) comando(s):

a) git pull
git checkout fix-issue-35
e, após atualizar o código, executar os comandos
git add .
git commit -m "Ajustes Maria"
git push origin fix-issue-35

b) git checkout fix-issue-35
git pull
e, após atualizar o código, executar os comandos
git commit -m "Ajustes Maria"
git push -u fix-issue-35

c) git pull
git checkout -b fix-issue-35 origin/fix-issue-35
e, após atualizar o código, executar os comandos
git add .
git commit -m "Ajustes Maria"
git push

d) git pull fix-issue-35
e, após atualizar o código, executar os comandos
git add *
git commit -m "Ajustes Maria"
git push fix-issue-35

e) git pull
e, após atualizar o código, executar os comandos
git commit -m "Ajustes Maria"
git push fix-issue-35

Comentários:

Bom, vamos ao fluxograma que Maria deveria seguir:

- Trazer o código para o repositório local com ***git pull***
- Mudar para o *branch*, verificando se o *branch* baixado está atualizado e íntegro, comparando-o com o original, com ***git checkout -b fix-issue-35 origin/fix-issue-35***
- Após as atualizações, adicionar ao repositório todas as modificações com ***git add***
- Commitar as alterações com ***git commit -m "Mensagem"***
- Empurrar as alterações para o repositório, com ***git push fix-issue-35*.**

A alternativa que traz corretamente esse fluxograma é a letra C.

Gabarito: Letra C

---

16. (FGV/TJ TO/2022) O técnico em informática José está desenvolvendo o software TJTOPlugin com o apoio da ferramenta de versionamento Git. José criou o branch local pluginConnector e efetuou alguns commits neste branch, mas não replicou os commits em um repositório remoto.
A fim de replicar os commits e criar o branch pluginConnector no repositório remoto origin, utilizando um único comando no terminal de comandos do sistema operacional, José deve executar o comando git com os argumentos:

a) mv pluginConnector origin;
b) diff pluginConnector origin;
c) merge origin/pluginConnector;

d) push origin pluginConnector;
e) remote add origin pluginConnector.

Comentários:

Para replicarmos o que criamos localmente no repositório remoto, precisamos "empurrar" o conteúdo para ele – para isso, usamos o comando *push*. O comando completo ficará *git push origin pluginConnector*

Gabarito: Letra D

---

17. (FGV/TRT 16/2022) Na ferramenta de versionamento Git, um branch de nome feature_n é criado ao executar o comando

a) git commit -m feature_n
b) git checkout -b feature_n
c) git remote -v feature_n
d) git push origin feature_n
e) Egit tag 1.1.0 feature_n

Comentários:

Para criar um *branch*, usualmente usamos o comando *git branch feature_n*. Alternativamente, podemos dar um *checkout* e, no mesmo comando, criar um *branch*, com o comando *git checkout -b feature_n*. A direita -b vem de --build, ou construir.

Gabarito: Letra B

---

18. (FCC/TRT 14/2022) Um analista está no diretório local ao qual o repositório Git remoto está vinculado. A partir desse diretório, para mover o arquivo tribunal.txt para a staging area, ele deve utilizar o comando

a) git move tribunal.txt to stage
b) git add tribunal.txt
c) git move tribunal.txt
d) git add tribunal.txt to stage
e) git stage tribunal.txt

Comentários:

O comando para mover determinado arquivo, no caso tribunal.txt, para a *staging area* é o *git add tribunal.txt*.

Gabarito: Letra B

---

19. (FCC/TRT 17/2022) No prompt de comandos do Windows, a partir da pasta do diretório local ao qual o diretório remoto do Git está vinculado, um analista digitou os comandos abaixo.

```
git add tribunal.txt
```

```
git status
git commit -m "Cria o arquivo tribunal.txt"
```

Considerando a existência do arquivo tribunal.txt no diretório e todas as condições ideais, para efetivar as alterações no repositório remoto, de forma que o arquivo possa ser visualizado por meio do link do repositório no navegador web, o analista deve digitar o comando

a) git persist
b) git merge
c) git push
d) git branch
e) git pull

Comentários:

Lembrem – sempre que quisermos jogar as alterações locais para o repositório, temos de empurrá-las, através do comando *git push*. Cuidado para não confundir com o comando para "puxar" os arquivos do repositório remoto, o *git pull*.

Gabarito: Letra C

---

20. (FCC/TRT 5/2022) Utilizando a ferramenta de versionamento Git, para mostrar, em linha de comando, a relação de arquivos com alterações pendentes para serem adicionados a um commit, um técnico deve utilizar o comando

a) git staged
b) git status
c) git remote show
d) git show files
e) git fetch files

Comentários:

Para encontrarmos arquivos pendentes, precisamos pesquisar pelo estado deles – usando o comando *git status*. Com o comando, iremos procurar por alterações com o estado *staged*, o que significa que essas alterações estão na *staging area*, prontas para serem *commitadas*.

Gabarito: Letra B

LISTA DE QUESTÕES

01. (CEBRASPE/CAU BR/2024) Quanto ao gerenciamento de configuração do *software* e aos serviços de mensageria, julgue o item a seguir.

Em um projeto de *software* que utilize a ferramenta Git para controle de versão, é recomendável que cada desenvolvedor trabalhe em sua própria *branch* local e faça *merge* com a *branch master* apenas quando o código estiver testado e revisado.

02. (CONSULPLAN/DPE PR/2024) O Git é um sistema de controle de revisão distribuído, rápido e escalável, com um conjunto de comandos que fornece operações de alto nível e acesso total aos componentes internos. Qual comando Git permite atualizar a versão local do repositório concurso a partir da versão remota?

a) git *checkout* concurso
b) git *pull origin* concurso
c) git *push origin* concurso
d) git *commit -m* "Atualização do repositório concurso"

03. (CONSULPLAN/DPE PR/2024) O comando *git stash* do *Git* é empregado para acumular as alterações em um outro diretório, ou seja, quando quiser gravar a condição atual do diretório ativo e do índice; porém, querendo voltar para um diretório funcional limpo. O comando salva as suas alterações locais e reverte o diretório de trabalho para coincidir ao *commit HEAD*. Sobre o comando *git stash do Git*, assinale a afirmativa **INCORRETA**.

a) *git stash clear*: realiza a remoção em ordem de pilha.
b) *git stash apply*: restaura as modificações de um stash.
c) *git stash list*: lista as entradas acumuladas (stash) que existem.
d) git stash show: lista todos os arquivos modificados no ultimo stash.

04. (CEBRASPE/ITAIPU/2024) Assinale a opção que apresenta o comando utilizado no Git para versionar o projeto com um pacote de alterações.

a) add
b) checkout
c) commit
d) clone
e) branch

05. (FGV/TJ AP/2024) A analista Cátia criou o branch tarefa10 em um repositório Git local. Logo após, Cátia alternou o branch de trabalho atual para o branch recém-criado, tarefa10. Estando no branch local tarefa10, Cátia precisou renomear o branch para t10_concluida, com um único comando da ferramenta git.

Para renomear o branch de trabalho atual com um único comando, Cátia executou no repositório Git local o comando do git:

a) mv t10_concluida;
b) reset t10_concluida;
c) rebase t10_concluida;
d) branch -m t10_concluida;
e) checkout -b t10_concluida.

06. (FGV/BBTS/2023) GIT é uma ferramenta utilizada para fazer controles de versões de projetos e seus arquivos.

Assinale a opção que apresenta os três possíveis estados em que os arquivos recém-criados ainda não foram submetidos a um snapshot.

a) commited, saved e shared.
b) commited, modified e staged.
c) consolidated, shared e logged.
d) consolidated, persisted e shared.
e) consolidated, confirmed e logged.

07. (AOCP/IF MA/2023) Qual comando pode ser utilizado dentro do GIT para listar o histórico de versões para um arquivo, incluindo mudanças de nome?

a) $ git show [arquivo]
b) $ git log
c) $ git log --follow [arquivo]
d) $ git rm [arquivo]
e) $ git stash drop

08. (VUNSEP/TJ RS/2023) Git é um sistema de controle de versões utilizado no desenvolvimento de código. Caso um bug tenha aparecido em um sistema que está sendo desenvolvido, com o apoio do Git é possível identificar qual commit foi responsável por este bug, com o auxílio de um comando.

Este comando é o:

a) git-bisect
b) git-blame
c) git-grep
d) git-status
e) git-tag

09. (VUNESP/TJ RS/2023) Na ferramenta de versionamento Git, dentre as alternativas a seguir, o nome do arquivo que permite definir tipos de arquivo que, intencionalmente, não devem ser rastreados no controle de versão é:

a) .ignore
b) .gitignore
c) .gituntracked
d) gitfile

e) makefile

10. (VUNESP/TJ RS/2023) Um dos sistemas de controle de versão mais populares para uso em desenvolvimento de software atualmente é o Git.

Com relação ao Git, assinale a alternativa que apresenta o comando que atualiza as referências locais com relações às remotas, informando ao Git local que ele deve obter as informações de metadados mais recentes do repositório original, mas não faz a integração e sobrescrita nos arquivos de trabalho locais.

a) status
b) pull
c) merge
d) fetch
e) remote

11. (CEBRASPE/MPE RO/2023) Caso se pretenda criar, no desenvolvimento de um novo código em certo projeto de software, um espaço no repositório Git que seja independente do principal, a fim de fazer alterações sem interferências no código principal, então isso poderá ser feito por meio do uso do recurso denominado

a) branch.
b) commit.
c) release.
d) rollback.
e) restore.

12. (VUNESP/TJM SP/2023) Git é um sistema de controle de versões utilizado no desenvolvimento de soluções de software.

Um usuário que deseja copiar o conteúdo de um repositório público em sua máquina pode fazer isso por meio do seguinte comando:

a) git-clone
b) git-copy
c) git-init
d) git-add
e) git-push

13. (VUNESP/TJM SP/2023) Um usuário está manipulando arquivos em um repositório Git. Para adicionar todos os novos arquivos no Git, ele pode utilizar o comando

a) git add -a
b) git add -i
c) git add -n
d) git add –u
e) git add -v

14. (FCC/PGE AM/2022) Um Técnico utilizou corretamente um comando git para modificar a mensagem do commit mais recente, ou seja, o último commit feito por ele no projeto. Trata-se do comando git

a) add merge.
b) push.
c) commit --amend.
d) add message.
e) checkout master.

15. (VUNESP/ALESP/2022) João criou uma nova ramificação (branch) no repositório Git chamada "fix-issue-35", submeteu o código por meio do comando commit e enviou as alterações para o repositório remoto por meio do comando push. Maria gostaria de contribuir com código nessa ramificação, então, para isso, ela precisa executar o(s) comando(s):

a) git pull
git checkout fix-issue-35
e, após atualizar o código, executar os comandos
git add .
git commit -m "Ajustes Maria"
git push origin fix-issue-35
b) git checkout fix-issue-35
git pull
e, após atualizar o código, executar os comandos
git commit -m "Ajustes Maria"
git push -u fix-issue-35
c) git pull
git checkout -b fix-issue-35 origin/fix-issue-35
e, após atualizar o código, executar os comandos
git add .
git commit -m "Ajustes Maria"
git push
d) git pull fix-issue-35
e, após atualizar o código, executar os comandos
git add *
git commit -m "Ajustes Maria"
git push fix-issue-35
e) git pull
e, após atualizar o código, executar os comandos
git commit -m "Ajustes Maria"
git push fix-issue-35

16. (FGV/TJ TO/2022) O técnico em informática José está desenvolvendo o software TJTOPlugin com o apoio da ferramenta de versionamento Git. José criou o branch local pluginConnector e efetuou alguns commits neste branch, mas não replicou os commits em um repositório remoto.
A fim de replicar os commits e criar o branch pluginConnector no repositório remoto origin, utilizando um único comando no terminal de comandos do sistema operacional, José deve executar o comando git com os argumentos:

a) mv pluginConnector origin;
b) diff pluginConnector origin;
c) merge origin/pluginConnector;
d) push origin pluginConnector;
e) remote add origin pluginConnector.

17. (FGV/TRT 16/2022) Na ferramenta de versionamento Git, um branch de nome feature_n é criado ao executar o comando

a) git commit -m feature_n
b) git checkout -b feature_n
c) git remote -v feature_n
d) git push origin feature_n
e) Egit tag 1.1.0 feature_n

18. (FCC/TRT 14/2022) Um analista está no diretório local ao qual o repositório Git remoto está vinculado. A partir desse diretório, para mover o arquivo tribunal.txt para a staging area, ele deve utilizar o comando

a) git move tribunal.txt to stage
b) git add tribunal.txt
c) git move tribunal.txt
d) git add tribunal.txt to stage
e) git stage tribunal.txt

19. (FCC/TRT 17/2022) No prompt de comandos do Windows, a partir da pasta do diretório local ao qual o diretório remoto do Git está vinculado, um analista digitou os comandos abaixo.

```
git add tribunal.txt
git status
git commit –m "Cria o arquivo tribunal.txt"
```

Considerando a existência do arquivo tribunal.txt no diretório e todas as condições ideais, para efetivar as alterações no repositório remoto, de forma que o arquivo possa ser visualizado por meio do link do repositório no navegador web, o analista deve digitar o comando

a) git persist
b) git merge
c) git push
d) git branch
e) git pull

20. (FCC/TRT 5/2022) Utilizando a ferramenta de versionamento Git, para mostrar, em linha de comando, a relação de arquivos com alterações pendentes para serem adicionados a um commit, um técnico deve utilizar o comando

a) git staged
b) git status
c) git remote show
d) git show files
e) git fetch files

GABARITO

GABARITO



1. Certo
2. Letra B
3. Letra A
4. Letra C
5. Letra D
6. Letra B
7. Letra C
8. Letra A
9. Letra B
10. Letra D
11. Letra A
12. Letra A
13. Letra A
14. Letra C
15. Letra C
16. Letra D
17. Letra B
18. Letra B
19. Letra C
20. Letra B

GitLab

Conceitos Gerais



O GitLab é uma plataforma completa para gestão do ciclo de vida do desenvolvimento de software, que integra controle de versão, gestão de projetos, revisão de código, integração contínua e entrega contínua (CI/CD), segurança e monitoramento em um único ambiente. Diferentemente de outras plataformas como o GitHub, que foca principalmente no repositório de código, o GitLab busca oferecer uma solução unificada para todas as etapas do desenvolvimento de software.

O GitLab foi criado com base no Git, um sistema de controle de versão distribuído que permite o rastreamento e a gestão de modificações no código-fonte ao longo do tempo. Isso significa que os desenvolvedores podem trabalhar de forma colaborativa, mantendo um histórico completo das alterações e garantindo a integridade do código.

Algumas funcionalidades oferecidas pelo GitLab incluem:

- Gerenciamento de Repositórios: Permite a criação e administração de repositórios Git com controle de acesso, revisão de código e merge requests.
- Integração Contínua e Entrega Contínua (CI/CD): Automatiza a execução de testes, compilação e implantação de aplicações.
- Gestão de Projetos: Oferece ferramentas como quadros Kanban, issues, milestones e wikis para auxiliar no planejamento e execução de projetos.
- Segurança e Compliance: Possui análises de segurança embutidas, como SAST (Static Application Security Testing), DAST (Dynamic Application Security Testing) e verificação de dependências para garantir a conformidade dos projetos.
- Monitoramento e Observabilidade: Integração com ferramentas de monitoramento para acompanhamento do desempenho das aplicações em produção.
- Automatização e Integração: Suporte para pipelines de DevOps, GitOps e integração com outras ferramentas como Kubernetes, Jira e Slack.

O GitLab atua de ponta a ponta, separando o ciclo de desenvolvimento em diferentes etapas:

- Planejamento
- Criação
- Verificação
- Segurança
- Release
- Monitoramento

02648216103 - HENRIQUE AMORIM BARRETO



Iremos focar essa aula no GitLab CI/CD – que controla a verificação, implementando a pipeline de Integração Contínua/Entrega Contínua.

(AOCP/IF MA/2023) A respeito do GitLab, uma plataforma de controle de versão distribuído e colaboração de código, assinale a alternativa correta.

a) GitLab é um fork do GitHub e, portanto, todas as funcionalidades presentes no GitHub estão disponíveis no GitLab.
b) GitLab é uma plataforma de gerenciamento de projetos ágeis e não possui integração com sistemas de controle de versão.
c) GitLab é um software de controle de versão distribuído e, ao contrário do GitHub, só pode ser usado on-premises (instalado no próprio servidor do cliente).
d) GitLab oferece funcionalidades como controle de versão, integração contínua/distribuição contínua (CI/CD) e gerenciamento de projetos, e pode ser usado tanto em sua versão SaaS quanto on-premises.
e) GitLab é uma alternativa ao Git e, portanto, não é compatível com repositórios Git existentes.

Comentários:

Vamos analisar as alternativas.

a) Errado. O GitLab não é um fork do GitHub, mas sim uma plataforma desenvolvida independentemente. Embora ambas tenham funcionalidades semelhantes, há diferenças em recursos e modelos de hospedagem.
b) Errado. O GitLab possui integração nativa com o Git, permitindo versionamento de código e colaboração entre desenvolvedores. Além disso, ele inclui ferramentas para gerenciamento ágil de projetos.
c) Errado. O GitLab pode ser instalado on-premises, mas também está disponível na nuvem (SaaS), ao contrário do que a alternativa sugere.
d) Certo. O GitLab oferece controle de versão, CI/CD e gerenciamento de projetos. Além disso, pode ser usado tanto em versão SaaS (na nuvem) quanto em versão self-hosted (on-premises).
e) Errado. O GitLab não é uma alternativa ao Git, mas sim uma plataforma que utiliza o Git como sistema de controle de versão, sendo totalmente compatível com repositórios Git.

Portanto, correta a letra D. *(Gabarito: Letra D)*

Configuração de Ambiente

O primeiro passo para trabalhar com o GitLab CI/CD é garantir que temos um repositório no GitLab. Após a devida configuração no repositório, podemos usar o *shell* para criarmos um clone do projeto e começarmos a trabalhar

```sh
git clone https://gitlab.com/usuario/meu-projeto.git
cd meu-projeto
```

O GitLab CI/CD utiliza um arquivo de configuração chamado `.gitlab-ci.yml`, localizado na raiz do repositório. Esse arquivo define os estágios do pipeline, os *jobs* que serão executados e as regras para acionamento de cada etapa.

Exploraremos ele mais à frente quando tivermos um conhecimento melhor sobre os componentes do GitLab, mas uma estrutura geral pode seguir a seguinte:

```yaml
stages:
  - build
  - test
  - deploy

build-job:
  stage: build
  script:
    - echo "Compilando a aplicação..."

test-job:
  stage: test
  script:
    - echo "Executando os testes..."

deploy-job:
  stage: deploy
  script:
    - echo "Realizando o deploy..."
```

Esse pipeline simples define três estágios (build, test e deploy), cada um contendo um *job* que imprime uma mensagem no terminal.

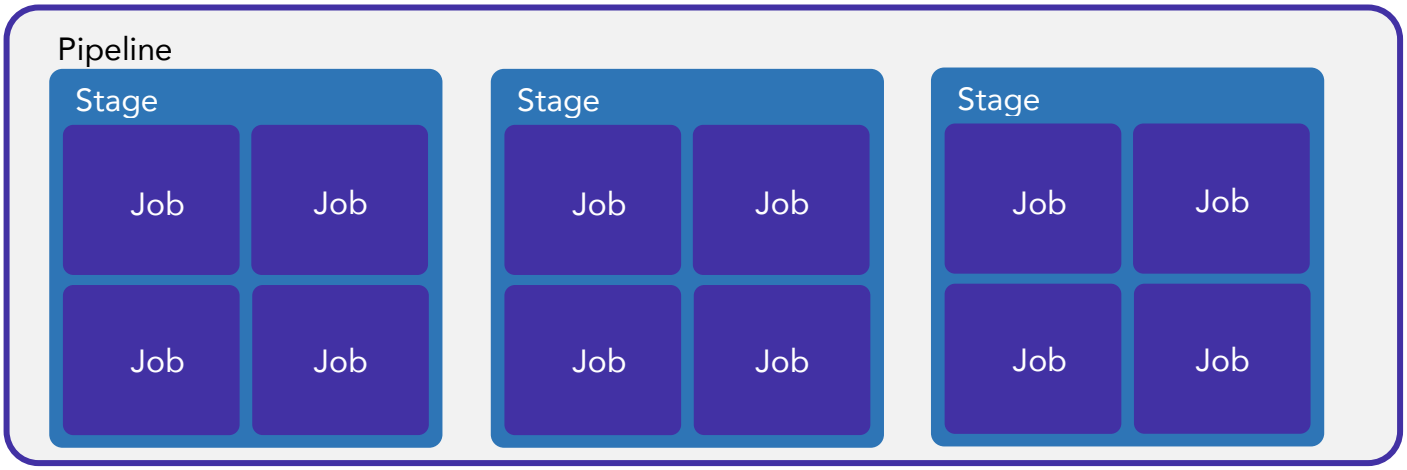
Em seguida, vamos abordar essas fases, o conceito de *runner* e outros elementos que são importantes para o entendimento da ferramenta.

Componentes

Podemos definir vários componentes dentro do GitLab para formar o ciclo CI/CD de uma aplicação. Vamos explorar esses componentes a partir de agora.

Pipeline

A **pipeline** é a estrutura central do GitLab CI/CD. Ela representa um conjunto de processos organizados que são executados em resposta a eventos no repositório, como *commits* ou *merges*. Cada pipeline pode conter múltiplos estágios (*stages*), que por sua vez agrupam *jobs* que podem rodar em paralelo ou em sequência, dependendo da configuração.



Estágios

Os **estágios** definem a sequência de execução dos *jobs* dentro do pipeline. Cada pipeline pode conter múltiplos estágios, sendo que os *jobs* dentro de um mesmo estágio são executados em paralelo (se houver runners disponíveis), e a execução do pipeline só avança para o próximo estágio quando todos os *jobs* do estágio atual forem concluídos com sucesso.

A estrutura básica de estágios é definida no arquivo `.gitlab-ci.yml` com a palavra-chave *stages*:

```yaml
stages:
  - build
  - test
  - deploy
```

Cada *job* dentro do pipeline deve estar associado a um estágio específico. Se um estágio falhar, os estágios subsequentes não serão executados, interrompendo o pipeline.

02648216103 - HENRIQUE AMORIM BARRETO

Jobs

Os *jobs* são as unidades individuais de execução dentro de cada estágio. Eles contêm as instruções que serão executadas pelos runners. Cada *job* no `.gitlab-ci.yml` precisa de:

- Um nome (por exemplo, build-*job*).
- Um estágio associado (stage: build).
- Um conjunto de comandos a serem executados (script).
- Opcionalmente, podem ter regras condicionais, variáveis e dependências.

Veja um exemplo:

yaml

```yaml
meu-job:
  stage: build
  script:
    - echo "Executando comandos..."
    - make build
```

Os *jobs* podem ser configurados de várias formas para atender a diferentes necessidades dentro do pipeline.

Podemos ter a **execução paralela** de *Jobs*. Isso ocorre quando temos definições de um conjunto de *Jobs* para mais de um *stage* ao mesmo tempo. Além disso, é possível limitar a execução de *Jobs* paralelos incluindo a diretiva `parallel`.

yaml

```yaml
stages:
  - build
  - test
  - deploy

build-job-1:
  stage: build
  script:
    - echo "Compilando código A"

build-job-2:
  stage: build
  script:
    - echo "Compilando código B"
```

Também é possível a implementação de execuções condicionais. Para isso, usamos três diretivas – `only` e `except`, que, apesarem de serem legado, ainda funcionam, e `rules`.

A diretiva **only** define em quais branches um *job* deve rodar. Então, se alocarmos a diretiva `only: -main`, esse *job* só será realizado quando estivermos na *branch main*. Já a diretiva **except** tem o caminho oposto, definindo quando *não* executaremos um *job*. Por exemplo, o `except: -tags` não executará commits se estiverem com tags.

Já a diretiva **rules** permite que trabalhemos condições mais flexíveis e avançadas. Nela, podemos elencar diversas condicionais que podem atuar cumulativamente.

```yaml
deploy-job:
  stage: deploy
  script:
    - echo "Realizando deploy..."
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'  # Só roda na branch main
    - if: '$CI_COMMIT_TAG'  # Ou se for um commit com tag
```

(FGV/TRF 1/2024) Considere o seguinte arquivo .gitlab-ci.yml, que define um pipeline no GitLab CI/CD:

```
workflow:
rules:
  - if: $CI_PIPELINE_SOURCE == "web"
  - if: $CI_PIPELINE_SOURCE == "push"

stages:
  - primeiro

primeiro:
  stage: primeiro
  script:
    - echo "Executando..."
```

O pipeline do GitLab CI/CD descrito acima:

a) não será executado, pois um *job* não pode ter o mesmo nome de um stage;
b) não será executado, pois a palavra-chave workflow é inválida;
c) será executado apenas no clique do botão Run Pipeline na interface do GitLab e nos eventos de push de branches ou de tags;

02648216103 - HENRIQUE AMORIM BARRETO

d) será executado apenas nos acionamentos via API do GitLab, e nos eventos de push de branches, mas não nos de tags;
e) será executado apenas nos acionamentos via WebHooks registrados no GitLab, e nos eventos de push de branches, mas não nos de tags.

Comentários:

O trecho workflow → rules contém as seguintes regras:

```
rules:
  - if: $CI_PIPELINE_SOURCE == "web"
  - if: $CI_PIPELINE_SOURCE == "push"
```

Isso significa que o pipeline será executado quando a origem do pipeline for:

- "web" → quando acionado manualmente pelo botão "Run Pipeline" na interface do GitLab.
- "push" → quando um push é realizado (pode ser para branches ou tags).

Portanto, correta a letra C. *(Gabarito: Letra C)*

Também é possível que sejam definidas dependências entre diferentes *jobs*, com a diretiva **needs**. O comportamento padrão é a leitura sequencial das instruções do YAML, mas essa diretiva permite que alteremos a execução para que dependências sejam executadas antes do seu respectivo estágio começar, ou após a execução de um *job* anterior.

yaml

```yaml
stages:
  - build
  - test

build-job:
  stage: build
  script:
    - echo "Construindo código"

test-job:
  stage: test
  needs:
    - build-job  # Esse job só será executado quando build-job terminar
  script:
    - echo "Rodando testes"
```

Também temos outras diretivas importantes que ajudam a configurar os *jobs*. Vamos reunir todas numa tabela-resumo.

Diretiva	Descrição
parallel	Permite executar múltiplas instâncias de um mesmo *job* simultaneamente, delimitando um limite máximo para instâncias concorrentes.
only	Define as condições em que um *job* deve ser executado com base na branch, tags ou eventos do pipeline.
except	Define as condições em que um *job* não deve ser executado, evitando rodar em branches ou eventos específicos.
rules	Substitui only e except, permitindo definir regras mais flexíveis para a execução de *jobs*, baseadas em expressões condicionais.
needs	Define dependências entre *jobs*, permitindo que um *job* seja executado antes do estágio ao qual pertence, se necessário.
retry	Especifica o número de vezes que um *job* pode ser reexecutado automaticamente em caso de falha.
allow_failure	Permite que um *job* falhe sem interromper o pipeline, útil para verificações opcionais como linting ou testes experimentais.
when	Controla o momento da execução do *job*. Pode ser configurado como on_success, on_failure, always, manual, entre outros.

Scripts

Os scripts definem os comandos que serão executados em um ambiente de CI/CD, como compilação, testes, deploys e outras tarefas automatizadas. No arquivo `.gitlab-ci.yml`, cada *job* pode conter um bloco script, onde os comandos são escritos da mesma forma que seriam executados em um terminal.

```yaml
meu-job:
  stage: build
  script:
    - echo "Iniciando build..."
    - make build
```


02648216103 - HENRIQUE AMORIM BARRETO

Runners

Os runners são responsáveis por executar os *jobs* definidos nos pipelines. Eles atuam como "máquinas de trabalho" que processam os comandos especificados no `.gitlab-ci.yml`, permitindo a automação de builds, testes e *deploys*. O *runner* é um serviço independente, que pode ser instalado em servidores locais, máquinas virtuais, containers Docker ou até mesmo em clusters Kubernetes. Além disse, os runners podem ser compartilhados entre projetos ou configurados para projetos específicos.

O GitLab oferece três tipos principais de runners:

- Shared Runners (Runners Compartilhados): fornecidos pelo próprio GitLab e podem ser usados por múltiplos projetos dentro da mesma instância. Recomendados para projetos de menor escala ou quando não há necessidade de runners dedicados.
- Specific Runners (Runners Específicos): registrados para um projeto ou grupo específico. Melhor opção para pipelines que exigem configurações personalizadas (exemplo: acesso a banco de dados interno).
- Group Runners (Runners de Grupo): São compartilhados entre todos os projetos dentro de um grupo GitLab. Úteis quando há múltiplos projetos com necessidades semelhantes.

yaml

```yaml
group-runner:
  image: ubuntu:latest
  script:
    - echo "Executando no runner do grupo"
```

Para definir *como* e *onde* os *Jobs* serão executados, usamos os executores. Eles determinam o ambiente no qual os scripts do `.gitlab-ci.yml` serão processados, podendo ser nativamente na máquina local, dentro de containers Docker, em máquinas virtuais ou até em clusters Kubernetes. Os principais executores são:

- Executor Shell: executa os comandos diretamente no sistema operacional do host onde o runner está instalado. Esse é o executor mais simples, sem isolamento entre *jobs*. Usando quando os *jobs* precisam rodar diretamente no sistema operacional e em builds simples.
- Executor Docker: executa cada *job* dentro de um container, garantindo isolamento entre execuções. É uma das opções mais usadas no GitLab CI/CD.
- Executor Docker+Machine: similar ao Docker, mas cria e destrói máquinas virtuais automaticamente para cada *job*. Ele usa Docker Machine para provisionar os ambientes.
- Executor Kubernetes: executa cada *job* dentro de um pod Kubernetes, permitindo escalabilidade e integração com clusters.
- Executor VirtualBox: cria uma máquina virtual e executa os *jobs* dentro dela.

02648216103 - HENRIQUE AMORIM BARRETO

- **Executor SSH:** permite rodar *jobs* em servidores remotos via conexão SSH. Ideal para rodar *jobs* em servidores remotos sem instalar um runner.

(FGV/TRF 1/2024) O analista Maicon deve configurar um novo runner, chamado RunM, no GitLab CI/CD do TRF1. Os *jobs* executados no RunM devem ser capazes de interagir de forma plena com um dispositivo externo conectado fisicamente ao host do runner. O escalonamento do RunM pelo GitLab deve ser fixo, e não automático. Maicon deve escolher o executor de runner do GitLab que exige a menor quantidade de customização necessária para a execução de RunM.

O executor de runner que Maicon deve escolher é:

a) shell;
b) docker;
c) parallels;
d) instance;
e) virtualbox.

Comentários:

O analista Maicon precisa configurar um runner no GitLab CI/CD que interaja plenamente com um dispositivo externo conectado fisicamente ao host. Além disso, o escalonamento deve ser fixo, e não automático, e a solução escolhida deve exigir o mínimo de customização possível. A forma mais indicada seria o executor *shell*. *(Gabarito: Letra A)*

Variables

As **variáveis**, ou *variables*, permitem armazenar valores configuráveis, como credenciais, configurações de ambiente e parâmetros personalizados para os *jobs*. No GitLab CI/CD, as variáveis podem ser definidas de diferentes formas e acessadas diretamente nos scripts do `.gitlab-ci.yml`.

yaml

```yaml
variables:
  AMBIENTE: "produção"
  API_URL: "https://api.meusistema.com"

deploy-job:
  stage: deploy
  script:
    - echo "Deployando no ambiente: $AMBIENTE"
    - curl -X GET "$API_URL"
```

Temos alguns tipos diferentes de variáveis, e diferentes locais onde elas são configuradas. As variáveis do arquivo de configuração, que são declaradas diretamente no `.gitlab-ci.yml` para cada pipeline, se tornam globais para cada pipeline.

Temos também variáveis de ambiente do GitLab, que são um conjunto de variáveis pré-definidas, que podem ser usadas diretamente dentro dos scripts. Alguns exemplos incluem:

Variável	Descrição
$CI_COMMIT_SHA	Hash do commit que acionou o pipeline.
$CI_COMMIT_BRANCH	Nome da branch do commit atual.
$CI_PIPELINE_ID	ID único do pipeline atual.
$CI_*JOB*_ID	ID do *job* em execução.
$CI_PROJECT_NAME	Nome do repositório GitLab.
$CI_COMMIT_TAG	Nome da tag, se o commit for uma tag.

Artifacts

Os artifacts são arquivos gerados por um *job* que podem ser armazenados e utilizados por outros *jobs* dentro do mesmo pipeline. Eles são particularmente úteis para compartilhar binários, logs, relatórios de testes, pacotes de compilação ou qualquer outro artefato necessário para as etapas seguintes do pipeline.

De forma geral, os *artifacts* permitem que arquivos gerados durante a execução de um *job* sejam:

- Salvos temporariamente no GitLab, permitindo download pela interface web.
- Compartilhados entre *jobs* subsequentes, permitindo que um *job* utilize arquivos gerados por outro.
- Expirados automaticamente após um período de tempo configurado.

Para definí-los, usamos a diretiva `artifacts`.

```yaml
build-job:
  stage: build
  script:
    - echo "Compilando código..."
    - mkdir build
    - touch build/app.bin
  artifacts:
    paths:
      - build/
    expire_in: 1h
```

Nesse exemplo:

- O *job* build-*job* cria um diretório build/ e um arquivo app.bin.
- O diretório build/ será salvo como um artifact.
- O arquivo ficará disponível para download na interface do GitLab por 1 hora (expire_in: 1h).

Além disso, os artifacts podem ser categorizados em duas categorias: Arquivos de build (paths), usados para armazenar binários, logs e dependências, e Relatórios (reports), específicos para integração com testes e segurança.

Caching

O caching permite armazenar arquivos e dependências entre execuções de pipelines, reduzindo tempo de build e consumo de recursos. Isso é especialmente útil para evitar o download repetitivo de bibliotecas, pacotes ou compilações intermediárias.

Embora pareça semelhante aos artifacts, o cache tem um propósito diferente: enquanto os artifacts são usados para compartilhar arquivos entre *jobs* do mesmo pipeline, o cache é projetado para manter arquivos entre diferentes execuções de pipeline.

A definição é feita no .gitlab-ci.yml:

```yaml
cache:
  paths:
    - node_modules/
```

Services

O services é um recurso que permite rodar contêineres auxiliares durante a execução de um *job*. Isso é útil quando precisamos de bancos de dados, servidores de cache, APIs simuladas ou qualquer outro serviço necessário para testar ou construir uma aplicação. Os services são frequentemente usados com o executor Docker, pois cada *job* é executado dentro de um container isolado, e os serviços fornecem suporte adicional a esses *jobs*.

Esses contêineres auxiliares podem rodar:

- Bancos de dados (MySQL, PostgreSQL, MongoDB)
- Servidores de cache (Redis, Memcached)
- Mensageria (RabbitMQ, Kafka)
- APIs auxiliares para testes

Importante destacar que os *services* são efêmeros, iniciados antes do *job* começar e desligados automaticamente após a execução do *job*. Veja um exemplo usando PostgreSQL:

yaml

```yaml
test-job:
  image: node:18
  stage: test
  services:
    - postgres:14
  variables:
    POSTGRES_DB: testdb
    POSTGRES_USER: user
    POSTGRES_PASSWORD: password
  script:
    - echo "Esperando o banco de dados..."
    - sleep 10
    - psql -h postgres -U user -d testdb -c "SELECT 'Banco pronto!'"
```

Merge

Merge é o processo de **integração de mudanças de uma branch para outra**, geralmente da branch de desenvolvimento para a branch principal (como main ou master). Esse processo é gerenciado por meio das **Merge Requests (MRs)**, que oferecem um mecanismo seguro e colaborativo para revisar, testar e aprovar alterações antes de integrá-las ao código principal.

O processo de MR permite:

- Revisão de código antes da fusão.
- Execução automática de testes via CI/CD.
- Histórico e rastreabilidade das mudanças.
- Aprovação de mudanças por membros do time.

Temos diferentes estratégia de Merge implementado a partir do Git Lab.

Variável	Descrição
Merge Commit (Padrão)	Cria um commit de merge preservando o histórico das branches.
Fast-Forward Merge	Move a branch de destino diretamente para a branch de origem (sem commit de merge).
Squash and Merge	Junta todos os commits em um único antes do merge.
Rebase and Merge	Reaplica os commits da branch de origem sobre a branch de destino.

Cada merge é escrito de diferentes formas e tem suas características, veja:

Merge padrão:

- Mantém o histórico completo da branch.
- Cria um commit extra para registrar o merge.

```sh
git merge feature/nova-funcionalidade
```

Fast-Forward Merge:

- Apenas avança a branch de destino se não houver commits divergentes.
- Mantém um histórico linear, sem commits extras

```sh
git merge --ff feature/nova-funcionalidade
```

Squash and Merge:

- Junta todos os commits em um único commit antes do merge.
- Mantém o histórico mais limpo.

```sh
git rebase -i main
git merge --squash feature/nova-funcionalidade
```

Rebase and Merge:

- Aplica os commits da branch feature um por um sobre a branch main.
- Evita commits de merge desnecessários.

```sh
git rebase main
git merge feature/nova-funcionalidade
```

(FGV/TCE SP/2023) A analista Jéssica administra o Gitlab do TCE SP. Jéssica atribuiu ao projeto TCEMóvel a estratégia de merge na qual, após feito o merge, o hash do último commit do branch de destino torna-se idêntico ao hash do último commit do branch de origem. A estratégia escolhida por Jéssica visa preservar a quantidade e a linearidade do histórico de commits, exigindo, porém, que o branch de origem esteja atualizado em relação ao destino para a efetuação do merge.

Jéssica atribuiu para o TCEMóvel a estratégia de merge:

a) merge commit;
b) three-way merge;
c) squash and merge;
d) fast-forward merge;
e) semi-linear merge commit.

Comentários:

Queremos uma abordagem de merge que "visa preservar a quantidade e a linearidade do histórico de commits". A abordagem mais indicada é a de fast-forward merge, que só irá avançar a branch de destino se não houver commits divergentes. *(Gabarito: Letra D)*

QUESTÕES COMENTADAS

01. (FGV/TRF 1/2024) O analista Maicon deve configurar um novo runner, chamado RunM, no GitLab CI/CD do TRF1. Os jobs executados no RunM devem ser capazes de interagir de forma plena com um dispositivo externo conectado fisicamente ao host do runner. O escalonamento do RunM pelo GitLab deve ser fixo, e não automático. Maicon deve escolher o executor de runner do GitLab que exige a menor quantidade de customização necessária para a execução de RunM.

O executor de runner que Maicon deve escolher é:

a) shell;
b) docker;
c) parallels;
d) instance;
e) virtualbox.

Comentários:

Maicon precisa configurar um runner no GitLab CI/CD que interaja plenamente com um dispositivo externo conectado fisicamente ao host. Além disso, o escalonamento deve ser fixo, e não automático, e a solução escolhida deve exigir o mínimo de customização possível.

Vamos analisar as alternativas considerando os requisitos:

a) Certo. O executor shell é a melhor opção, pois executa os jobs diretamente no sistema operacional do host, sem isolamento adicional. Isso permite acesso direto a dispositivos conectados fisicamente, sem necessidade de configuração extra.
b) Errado. O executor docker executa jobs em contêineres isolados, o que dificulta a interação direta com dispositivos externos conectados ao host.
c) Errado. O executor parallels é utilizado para rodar máquinas virtuais macOS dentro do ambiente do runner, não sendo adequado para interação direta com hardware externo.
d) Errado. O executor instance não é uma opção válida no GitLab CI/CD.
e) Errado. O executor virtualbox roda jobs dentro de máquinas virtuais, o que cria uma camada de isolamento que dificulta o acesso direto a dispositivos externos conectados ao host.

Portanto, correta a letra A.

Gabarito: Letra A

---

02. (FGV/TRF 1/2024) Considere o seguinte arquivo .gitlab-ci.yml, que define um pipeline no GitLab CI/CD:

```
workflow:
rules:
  - if: $CI_PIPELINE_SOURCE == "web"
  - if: $CI_PIPELINE_SOURCE == "push"

stages:
  - primeiro

primeiro:
  stage: primeiro
  script:
    - echo "Executando..."
```

O pipeline do GitLab CI/CD descrito acima:

a) não será executado, pois um job não pode ter o mesmo nome de um stage;
b) não será executado, pois a palavra-chave workflow é inválida;
c) será executado apenas no clique do botão Run Pipeline na interface do GitLab e nos eventos de push de branches ou de tags;
d) será executado apenas nos acionamentos via API do GitLab, e nos eventos de push de branches, mas não nos de tags;
e) será executado apenas nos acionamentos via WebHooks registrados no GitLab, e nos eventos de push de branches, mas não nos de tags.

Comentários:

O trecho workflow → rules contém as seguintes regras:

```
rules:
  - if: $CI_PIPELINE_SOURCE == "web"
  - if: $CI_PIPELINE_SOURCE == "push"
```

Isso significa que o pipeline será executado quando a origem do pipeline for:

- "web" → quando acionado manualmente pelo botão "Run Pipeline" na interface do GitLab.
- "push" → quando um push é realizado (pode ser para branches ou tags).

Portanto, correta a letra C.

Gabarito: Letra C

03. (CEBRASPE/TSE/2024) A respeito de ferramentas de controle de versão e ferramentas de integração contínua, julgue o item subsequente.

Um arquivo .gitlab-ci.yml no GitLab é usado para definir os stages e jobs do pipeline, além de especificar variáveis de ambiente, configurar regras de deploy e definir condições para a execução dos jobs.

Comentários:

Perfeito! O arquivo .gitlab-ci.yml é essencial no GitLab CI/CD e tem a função de definir os stages e jobs do pipeline, permitindo a automação de processos como build, teste e deploy. Com ele, podemos:

- Especificar variáveis de ambiente, que podem ser usadas dentro dos jobs.
- Configurar regras de deploy, definindo quando e como os jobs devem ser executados.
- Definir condições para a execução dos jobs, utilizando rules, only, except, when, entre outros parâmetros.

Portanto, correto o gabarito.

Gabarito: Certo

---

04. (CEBRASPE/SEBRAE/2024) Certo analista está configurando um webhook no GitLab para integrá-lo a um serviço externo que processa notificações de commits em uma instância de administração. Nesse caso, para garantir o funcionamento correto, o tamanho máximo permitido, por padrão, para o payload do webhook é de

a) 10 MB.
b) 25 MB.
c) 50 MB.
d) 100 MB.

Comentários:

Ao configurar um webhook no GitLab para integrar com um serviço externo que processa notificações de commits, é importante considerar o tamanho máximo permitido para o payload do webhook. De acordo com a documentação oficial do GitLab, o tamanho máximo padrão para o payload de um webhook é de 25 MB.

Gabarito: Letra B

---

05. (CEBRASPE/SEBRAE/2024) No GitLab, as permissões determinam as ações que um usuário pode realizar em um repositório. Para que um usuário possa criar uma nova branch de um repositório, ele precisa ter, pelo menos, permissões de

a) dono.
b) convidado.
c) desenvolvedor.
d) relator.

Comentários:

No GitLab, as permissões determinam o que um usuário pode fazer dentro de um repositório. Para criar uma nova branch, é necessário ter um nível mínimo de permissões de **desenvolvedor**. Quanto aos outros níveis:

- Dono: tem controle total sobre o projeto, mas não é necessário ter esse nível pra criar branches;
- Convidade: tem acesso apenas à interface do projeto, não podendo interagir com o código;
- Relator: pode visualizar e comentar o código, mas não pode criar branches, fazer push, entre outras modificações

Portanto, correta a letra C.

Gabarito: Letra C

---

06. (CEBRASPE/SEBRAE/2024) Assinale a opção que corresponde ao arquivo de configuração padrão para pipelines no GitLab CI/CD.

a) .build-settings.yml
b) .ci-config.yml
c) .gitlab-ci.yml
d) .pipeline.yml

Comentários:

O arquivo de configuração para o GitLab CI/CD é o `.gitlab-ci.yml`.

Gabarito: Letra C

---

07. (UFMT/UFMT/2024) Marque a alternativa que descreve para que serve a tag stages do pipeline em GitLab.

02648216103 - HENRIQUE AMORIM BARRETO

a) É utilizada para definir a ordem das diferentes etapas do pipeline de CI/CD.
b) É utilizada para definir quais branches do repositório serão utilizados para executar o pipeline.
c) É utilizada para definir variáveis de ambiente que serão utilizadas pelos jobs do pipeline.
d) É utilizada para definir os scripts de deploy que serão executados pelo pipeline.

Comentários:

Vamos analisar as alternativas:

a) Certo. A tag stages define a ordem das etapas do pipeline, garantindo que os jobs sejam executados na sequência correta.
b) Errado. A definição de quais branches irão acionar o pipeline é feita com only, except ou rules, e não com stages.
c) Errado. A definição de variáveis de ambiente no GitLab CI/CD é feita com variables, não com stages.
d) Errado. Os scripts de deploy são definidos dentro dos jobs usando script: e não pela tag stages.

Portanto, o gabarito é a letra A.

Gabarito: Letra A

---

08. (CEBRASPE/CAU BR/2024) Julgue o item que se segue a respeito do Gitlab.

O Gitlab suporta o uso de snippets, que são trechos de código reutilizáveis e possíveis de serem compartilhados entre projetos.

Comentários:

O GitLab suporta o uso de snippets, que são pequenos trechos de código que podem ser armazenados e compartilhados dentro da plataforma. Eles permitem que os usuários reutilizem códigos, scripts e comandos, facilitando a colaboração entre equipes. Além disso, o GitLab oferece snippets pessoais, que são privados para cada usuário, e snippets de projeto ou grupo, que podem ser compartilhados com outros membros do repositório.

Gabarito: Certo

---

09. (CEBRASPE/CAU BR/2024) Julgue o item que se segue a respeito do Gitlab.

O Gitlab disponibiliza recursos nativos capazes de automatizar testes e verificar a qualidade do código, conhecidos como CI (*continuous integration*), sendo necessários recursos de terceiros para executar a etapa de CD *(continuous delivery*).

Comentários:

Errado! O GitLab disponibiliza recursos nativos tanto para CI (Continuous Integration) quanto para CD (Continuous Delivery e Continuous Deployment). Com a configuração correta do arquivo .gitlab-ci.yml, é possível definir pipelines completos, desde a fase de teste até o deploy para produção, sem depender de soluções de terceiros.

Gabarito: Errado

---

10. (FGV/TCE SP/2023) A analista Jéssica administra o Gitlab do TCE SP. Jéssica atribuiu ao projeto TCEMóvel a estratégia de merge na qual, após feito o merge, o hash do último commit do branch de destino torna-se idêntico ao hash do último commit do branch de origem. A estratégia escolhida por Jéssica visa preservar a quantidade e a linearidade do histórico de commits, exigindo, porém, que o branch de origem esteja atualizado em relação ao destino para a efetuação do merge.

Jéssica atribuiu para o TCEMóvel a estratégia de merge:

a) merge commit;
b) three-way merge;
c) squash and merge;
d) fast-forward merge;
e) semi-linear merge commit.

Comentários:

A estratégia de merge escolhida pela analista Jéssica garante que, após o merge, o hash do último commit do branch de destino seja idêntico ao hash do último commit do branch de origem. Além disso, a estratégia preserva a linearidade do histórico e exige que o branch de origem esteja atualizado.

Vamos analisar as alternativas para encontrar a correta.

a) Errado. O merge commit cria um novo commit de merge, preservando o histórico completo, mas não mantém a linearidade do histórico, já que introduz um commit extra para o merge.

02648216103 - HENRIQUE AMORIM BARRETO

b) Errado. O three-way merge é um tipo de merge commit, no qual o Git compara três estados do código (o branch base, o branch de origem e o branch de destino) para resolver diferenças. Isso não preserva a linearidade, pois também cria um commit de merge.
c) Errado. O squash and merge combina todos os commits do branch de origem em um único commit, simplificando o histórico, mas não mantém a estrutura original dos commits.
d) Certo. O fast-forward merge mantém a linearidade do histórico porque simplesmente avança o ponteiro do branch de destino para o commit do branch de origem, sem criar um commit de merge. Para isso, exige que o branch de origem esteja atualizado em relação ao destino, como descrito na questão.
e) Errado. O semi-linear merge commit é uma variação que mantém a linearidade, mas ainda pode gerar um commit de merge quando necessário.

Portanto, correta a letra D.

Gabarito: Letra D

---

11. (CEBRASPE/DATAPREV2023) A respeito do Gitlab, julgue o item a seguir.

O *runner* tem como função a execução de *pipelines* e o versionamento automático do código-fonte de um repositório.

Comentários:

O runner no GitLab tem como função executar pipelines, ou seja, processar os jobs definidos no arquivo .gitlab-ci.yml. Ele pode ser configurado para rodar em diferentes ambientes, como shell, Docker e máquinas virtuais, permitindo a automação de testes, builds e deploys.

Porém, um runner não realiza versionamento automático do código-fonte. O versionamento do código no GitLab é gerenciado pelo Git, e qualquer modificação no repositório precisa ser feita manualmente por um usuário via commits e pushes.

Gabarito: Errado

---

12. (CEBRASPE/DATAPREV/2023) A respeito do Gitlab, julgue o item a seguir.

O Gitlab suporta MR (*merge request*) que é capaz de mesclar a *branch* de origem com uma *branch* alvo.

Comentários:

Certo! No GitLab, o MR (Merge Request) é um recurso essencial para a colaboração no desenvolvimento de software. Ele permite que um desenvolvedor solicite a fusão (merge) de uma

branch de origem para uma branch de destino, garantindo a revisão do código antes da integração.

Além de permitir a mesclagem de branches, o Merge Request também pode incluir revisões de código, aprovações, verificações automatizadas via CI/CD, e opções como squash, rebase ou fast-forward merge.

Gabarito: Certo

---

13. (CEBRASPE/DATAPREV/2023) A respeito do Gitlab, julgue o item a seguir.

É possível visualizar *jobs* de um *pipeline* e executar operações como cancelar e tentar novamente se der erro, mas os registros de *log* não podem ser apagados, pois registram o comportamento da execução dos *jobs*.

Comentários:

No GitLab CI/CD, é possível visualizar os jobs de um pipeline, acompanhar sua execução e realizar ações como cancelar um job em andamento ou reexecutá-lo caso ocorra falha. Além disso, os logs de execução dos jobs não podem ser apagados, pois são mantidos como registro do comportamento da execução, garantindo transparência e rastreabilidade. Esses logs são essenciais para auditoria, depuração de erros e análise de falhas, sendo parte integrante do histórico do pipeline. Portanto, a afirmação está correta.

Gabarito: Certo

---

14. (AOCP/IF MA/2023) A respeito do GitLab, uma plataforma de controle de versão distribuído e colaboração de código, assinale a alternativa correta.

a) GitLab é um fork do GitHub e, portanto, todas as funcionalidades presentes no GitHub estão disponíveis no GitLab.
b) GitLab é uma plataforma de gerenciamento de projetos ágeis e não possui integração com sistemas de controle de versão.
c) GitLab é um software de controle de versão distribuído e, ao contrário do GitHub, só pode ser usado on-premises (instalado no próprio servidor do cliente).
d) GitLab oferece funcionalidades como controle de versão, integração contínua/distribuição contínua (CI/CD) e gerenciamento de projetos, e pode ser usado tanto em sua versão SaaS quanto on-premises.
e) GitLab é uma alternativa ao Git e, portanto, não é compatível com repositórios Git existentes.

Comentários:

Vamos analisar as alternativas.

a) Errado. O GitLab não é um fork do GitHub, mas sim uma plataforma desenvolvida independentemente. Embora ambas tenham funcionalidades semelhantes, há diferenças em recursos e modelos de hospedagem.
b) Errado. O GitLab possui integração nativa com o Git, permitindo versionamento de código e colaboração entre desenvolvedores. Além disso, ele inclui ferramentas para gerenciamento ágil de projetos.
c) Errado. O GitLab pode ser instalado on-premises, mas também está disponível na nuvem (SaaS), ao contrário do que a alternativa sugere.
d) Certo. O GitLab oferece controle de versão, CI/CD e gerenciamento de projetos. Além disso, pode ser usado tanto em versão SaaS (na nuvem) quanto em versão self-hosted (on-premises).
e) Errado. O GitLab não é uma alternativa ao Git, mas sim uma plataforma que utiliza o Git como sistema de controle de versão, sendo totalmente compatível com repositórios Git.

Portanto, correta a letra D.

Gabarito: Letra D

---

15. (AOCP/IF MA/2023) João está trabalhando em um projeto de desenvolvimento de software usando Git e GitLab como plataforma de colaboração. Ele criou um branch separado para implementar uma nova funcionalidade e, após concluir seu trabalho, deseja integrar suas alterações ao branch principal. Ele decide abrir um merge request (também chamado de pull request) para que suas alterações sejam revisadas por seus colegas de equipe antes de serem mescladas. Qual é o principal objetivo do merge request nesse contexto?

a) Iniciar um processo de revisão de código, permitindo que outros membros da equipe analisem e discutam as alterações propostas antes de integrá-las ao branch principal.
b) Fazer o backup das alterações no branch de trabalho, permitindo que outros membros da equipe as acessem e as modifiquem conforme necessário.
c) Criar um novo branch de trabalho que inclua as alterações propostas, permitindo que outros membros da equipe contribuam para a funcionalidade em desenvolvimento.
d) Forçar o merge das alterações no branch principal, garantindo que a funcionalidade seja imediatamente disponibilizada para todos os membros da equipe.
e) Comparar as alterações no branch de trabalho com o branch principal, identificando possíveis conflitos que possam surgir durante o merge.

Comentários:

Vamos analisar as alternativas.

a) Certo. O principal objetivo do Merge Request é iniciar um processo de revisão de código, permitindo que outros membros da equipe analisem, discutam e sugiram melhorias antes da integração ao branch principal. Esse processo ajuda a evitar problemas, melhora a qualidade do código e garante conformidade com os padrões do projeto.
b) Errado. O Merge Request não é um mecanismo de backup. O versionamento no Git já garante que as alterações estejam salvas e acessíveis no repositório remoto.
c) Errado. O Merge Request não cria um novo branch, ele apenas propõe a fusão das alterações do branch de origem no branch de destino.
d) Errado. O Merge Request não força o merge automaticamente. Ele permite que as alterações sejam revisadas antes da fusão, podendo até ser rejeitadas ou ajustadas conforme necessário.
e) Errado. Embora o Merge Request exiba diferenças entre branches e possíveis conflitos, esse não é seu principal objetivo. A revisão do código é a parte mais importante do processo.

Portanto, correta a letra A.

Gabarito: Letra A

---

16. (AOCP/IF MA/2023) No GitLab, qual é a característica fundamental do arquivo .gitlab-ci.yml, utilizado para configurar pipelines de Continuous Integration e Continuous Delivery (CI/CD)?

a) Arquivo de configuração no formato YAML que define as etapas do pipeline.
b) Arquivo de configuração no formato JSON que define as etapas do pipeline.
c) Arquivo binário que deve ser criado usando a interface web do GitLab.
d) Deve ser armazenado fora do repositório do projeto para garantir a segurança.
e) Deve ser criado manualmente pelo administrador do sistema para cada projeto.

Comentários:

O .gitlab-ci.yml é um arquivo de configuração para as pipelines do GitLab, que segue o formato YAML. Portanto, correta a letra A.

Gabarito: Letra A

---

17. (FGV/PGM Niterói/2023) A analista Ana configurou o pipeline MainPipe no GitLab CI/CD da PGM de Niterói. O MainPipe possui um único job, o BuildJob, que automatiza o processo de build da aplicação PGMApp. O build de PGMApp é realizado em um container Docker e demanda a execução de um banco de dados em outro container Docker. A fim de aproveitar os recursos de integração do GitLab CI/CD com o Docker, Ana especificou as imagens Docker necessárias ao build diretamente no arquivo .gitlab-ci.yml do MainPipe, no trecho associado ao BuildJob.

Logo, Ana especificou a imagem Docker para o build e a imagem Docker para o banco de dados, respectivamente, nas subseções do .gitlab-ci.yml:

a) image e services;
b) image e artifacts;
c) workflow e services;
d) workflow e artifacts;
e) artifacts e workflow.

Comentários:

Ana configurou o pipeline MainPipe no GitLab CI/CD, onde o BuildJob precisa rodar um container Docker para o build e um outro container para o banco de dados. No GitLab CI/CD, quando usamos Docker como executor, é possível especificar:

- A imagem principal do container onde o job será executado, que é feito na seção *image*.
- Imagens adicionais necessárias, como um banco de dados, para auxiliar na execução do job. Essa definição acontece no *services*.

Portanto, temos *image* e *services* – correta a letra A.

Gabarito: Letra A

---

18. (CEBRASPE/TRT 8/2022) O GitLab permite que os usuários executem verificações de segurança para garantir que as imagens do Docker não tenham nenhuma vulnerabilidade no ambiente. Essa funcionalidade é disponibilizada pelo recurso

a) Package Management.
b) Activity Stream.
c) Powerful Branching.
d) Container Scanning.
e) Auto DevOps.

Comentários:

Vamos às alternativas.

a) Errado. O Package Management permite gerenciar pacotes e dependências dentro do GitLab, mas não realiza verificação de vulnerabilidades em imagens Docker.
b) Errado. O Activity Stream fornece um histórico das ações realizadas no repositório, mas não está relacionado à segurança ou verificação de vulnerabilidades.

c) Errado. O Powerful Branching refere-se às funcionalidades avançadas de gerenciamento de branches no GitLab, mas não inclui verificações de segurança.
d) Certo. O Container Scanning é a funcionalidade do GitLab que permite verificar imagens Docker em busca de vulnerabilidades, ajudando a garantir a segurança do ambiente antes da implantação.
e) Errado. O Auto DevOps automatiza processos de CI/CD, incluindo deploy e monitoramento, mas não se limita à verificação de vulnerabilidades em imagens Docker.

Portanto, correta a letra D.

Gabarito: Letra D

---

19. (FCC/TRT 22/2022) Considere o recorte de um arquivo .gitlab-ci.yml que traz algumas opções de configuração de um pipeline GitLab CI/CD, nas seções que são separadas pelas global keywords.

```
stages:
  - sync
  - prepare
  - build-images

..I..:
  interruptible: true
  timeout: 90m

workflow:
# regras

variables:
# variáveis

..II.. :
  - local: .gitlab/ci/*.gitlab-ci.yml
  - remote: 'https://gitlab.com/gitlab-org/frontend/-/merge_request_pipelines.yml'
```

As global keywords I e II correspondem, correta e respectivamente, a

a) tags e directories
b) dependencies e environment
c) dependencies e pages
d) default e include
e) default e directories

Comentários:

No arquivo .gitlab-ci.yml, as global keywords são palavras-chave que definem configurações globais aplicáveis ao pipeline. Vamos analisar quais palavras-chave correspondem às marcações "..I.." e "..II..".

A primeira palavra-chave (..I..) contém as configurações:

```
interruptible: true
timeout: 90m
```

Esses parâmetros são usados dentro da seção default, que define configurações padrão para os jobs, como tempo limite (timeout) e comportamento em caso de interrupção (interruptible).

A segunda palavra-chave (..II..) contém:

```
- local: .gitlab/ci/*.gitlab-ci.yml
- remote: 'https://gitlab.com/gitlab-org/frontend/-/merge_request_pipelines.yml'
```

Isso indica a importação de arquivos de configuração, algo feito com a keyword include, permitindo a reutilização de configurações de outros arquivos .gitlab-ci.yml.

Portanto, correta a letra D.

Gabarito: Letra D

---

20. (FCC/TJ CE/2022) Para usar o GitLab CI/CD são necessários os seguintes passos iniciais:

– Certificar-se que haja runners disponíveis para executar os jobs. Se não houver um runner, instalar o GitLab Runner e registrar um runner para a instância, projeto ou grupo.
– Criar um arquivo I na raiz do repositório. Este arquivo é o local onde se definem os jobs CI/CD.
– Quando for feito o commit deste arquivo no repositório, o runner executará os jobs.

Preenche corretamente a lacuna I:

a) .gitlab-cd.yaml
b) .gitlab-cd.yml
c) .gitlab-cicd.yaml
d) .gitlab-cicd.yml
e) .gitlab-ci.yml

Comentários:

Questão tranquila – o arquivo na raiz do repositório, onde definimos os Jobs, é o `.gitlab-ci.yml`.

Gabarito: Letra E

LISTA DE QUESTÕES

01. (FGV/TRF 1/2024) O analista Maicon deve configurar um novo runner, chamado RunM, no GitLab CI/CD do TRF1. Os jobs executados no RunM devem ser capazes de interagir de forma plena com um dispositivo externo conectado fisicamente ao host do runner. O escalonamento do RunM pelo GitLab deve ser fixo, e não automático. Maicon deve escolher o executor de runner do GitLab que exige a menor quantidade de customização necessária para a execução de RunM.

O executor de runner que Maicon deve escolher é:

a) shell;
b) docker;
c) parallels;
d) instance;
e) virtualbox.

02. (FGV/TRF 1/2024) Considere o seguinte arquivo .gitlab-ci.yml, que define um pipeline no GitLab CI/CD:

```
workflow:
rules:
  - if: $CI_PIPELINE_SOURCE == "web"
  - if: $CI_PIPELINE_SOURCE == "push"

stages:
  - primeiro

primeiro:
  stage: primeiro
  script:
    - echo "Executando..."
```

O pipeline do GitLab CI/CD descrito acima:

a) não será executado, pois um job não pode ter o mesmo nome de um stage;
b) não será executado, pois a palavra-chave workflow é inválida;
c) será executado apenas no clique do botão Run Pipeline na interface do GitLab e nos eventos de push de branches ou de tags;
d) será executado apenas nos acionamentos via API do GitLab, e nos eventos de push de branches, mas não nos de tags;

02648216103 - HENRIQUE AMORIM BARRETO

e) será executado apenas nos acionamentos via WebHooks registrados no GitLab, e nos eventos de push de branches, mas não nos de tags.

03. (CEBRASPE/TSE/2024) A respeito de ferramentas de controle de versão e ferramentas de integração contínua, julgue o item subsequente.

Um arquivo .gitlab-ci.yml no GitLab é usado para definir os stages e jobs do pipeline, além de especificar variáveis de ambiente, configurar regras de deploy e definir condições para a execução dos jobs.

04. (CEBRASPE/SEBRAE/2024) Certo analista está configurando um webhook no GitLab para integrá-lo a um serviço externo que processa notificações de commits em uma instância de administração. Nesse caso, para garantir o funcionamento correto, o tamanho máximo permitido, por padrão, para o payload do webhook é de

a) 10 MB.
b) 25 MB.
c) 50 MB.
d) 100 MB.

05. (CEBRASPE/SEBRAE/2024) No GitLab, as permissões determinam as ações que um usuário pode realizar em um repositório. Para que um usuário possa criar uma nova branch de um repositório, ele precisa ter, pelo menos, permissões de

a) dono.
b) convidado.
c) desenvolvedor.
d) relator.

06. (CEBRASPE/SEBRAE/2024) Assinale a opção que corresponde ao arquivo de configuração padrão para pipelines no GitLab CI/CD.

a) .build-settings.yml
b) .ci-config.yml
c) .gitlab-ci.yml
d) .pipeline.yml

07. (UFMT/UFMT/2024) Marque a alternativa que descreve para que serve a tag stages do pipeline em GitLab.

a) É utilizada para definir a ordem das diferentes etapas do pipeline de CI/CD.
b) É utilizada para definir quais branches do repositório serão utilizados para executar o pipeline.
c) É utilizada para definir variáveis de ambiente que serão utilizadas pelos jobs do pipeline.
d) É utilizada para definir os scripts de deploy que serão executados pelo pipeline.

08. (CEBRASPE/CAU BR/2024) Julgue o item que se segue a respeito do Gitlab.

O Gitlab suporta o uso de snippets, que são trechos de código reutilizáveis e possíveis de serem compartilhados entre projetos.

09. (CEBRASPE/CAU BR/2024) Julgue o item que se segue a respeito do Gitlab.

O Gitlab disponibiliza recursos nativos capazes de automatizar testes e verificar a qualidade do código, conhecidos como CI (*continuous integration*), sendo necessários recursos de terceiros para executar a etapa de CD *(continuous delivery*).

10. (FGV/TCE SP/2023) A analista Jéssica administra o Gitlab do TCE SP. Jéssica atribuiu ao projeto TCEMóvel a estratégia de merge na qual, após feito o merge, o hash do último commit do branch de destino torna-se idêntico ao hash do último commit do branch de origem. A estratégia escolhida por Jéssica visa preservar a quantidade e a linearidade do histórico de commits, exigindo, porém, que o branch de origem esteja atualizado em relação ao destino para a efetuação do merge.

Jéssica atribuiu para o TCEMóvel a estratégia de merge:

a) merge commit;
b) three-way merge;
c) squash and merge;
d) fast-forward merge;
e) semi-linear merge commit.

11. (CEBRASPE/DATAPREV2023) A respeito do Gitlab, julgue o item a seguir.

O *runner* tem como função a execução de *pipelines* e o versionamento automático do código-fonte de um repositório.

12. (CEBRASPE/DATAPREV/2023) A respeito do Gitlab, julgue o item a seguir.

O Gitlab suporta MR (*merge request*) que é capaz de mesclar a *branch* de origem com uma *branch* alvo.

13. (CEBRASPE/DATAPREV/2023) A respeito do Gitlab, julgue o item a seguir.

É possível visualizar *jobs* de um *pipeline* e executar operações como cancelar e tentar novamente se der erro, mas os registros de *log* não podem ser apagados, pois registram o comportamento da execução dos *jobs*.

14. (AOCP/IF MA/2023) A respeito do GitLab, uma plataforma de controle de versão distribuído e colaboração de código, assinale a alternativa correta.

a) GitLab é um fork do GitHub e, portanto, todas as funcionalidades presentes no GitHub estão disponíveis no GitLab.
b) GitLab é uma plataforma de gerenciamento de projetos ágeis e não possui integração com sistemas de controle de versão.
c) GitLab é um software de controle de versão distribuído e, ao contrário do GitHub, só pode ser usado on-premises (instalado no próprio servidor do cliente).
d) GitLab oferece funcionalidades como controle de versão, integração contínua/distribuição contínua (CI/CD) e gerenciamento de projetos, e pode ser usado tanto em sua versão SaaS quanto on-premises.
e) GitLab é uma alternativa ao Git e, portanto, não é compatível com repositórios Git existentes.

15. (AOCP/IF MA/2023) João está trabalhando em um projeto de desenvolvimento de software usando Git e GitLab como plataforma de colaboração. Ele criou um branch separado para implementar uma nova funcionalidade e, após concluir seu trabalho, deseja integrar suas alterações ao branch principal. Ele decide abrir um merge request (também chamado de pull request) para que suas alterações sejam revisadas por seus colegas de equipe antes de serem mescladas. Qual é o principal objetivo do merge request nesse contexto?

a) Iniciar um processo de revisão de código, permitindo que outros membros da equipe analisem e discutam as alterações propostas antes de integrá-las ao branch principal.
b) Fazer o backup das alterações no branch de trabalho, permitindo que outros membros da equipe as acessem e as modifiquem conforme necessário.
c) Criar um novo branch de trabalho que inclua as alterações propostas, permitindo que outros membros da equipe contribuam para a funcionalidade em desenvolvimento.
d) Forçar o merge das alterações no branch principal, garantindo que a funcionalidade seja imediatamente disponibilizada para todos os membros da equipe.
e) Comparar as alterações no branch de trabalho com o branch principal, identificando possíveis conflitos que possam surgir durante o merge.

16. (AOCP/IF MA/2023) No GitLab, qual é a característica fundamental do arquivo .gitlab-ci.yml, utilizado para configurar pipelines de Continuous Integration e Continuous Delivery (CI/CD)?

a) Arquivo de configuração no formato YAML que define as etapas do pipeline.
b) Arquivo de configuração no formato JSON que define as etapas do pipeline.
c) Arquivo binário que deve ser criado usando a interface web do GitLab.
d) Deve ser armazenado fora do repositório do projeto para garantir a segurança.
e) Deve ser criado manualmente pelo administrador do sistema para cada projeto.

17. (FGV/PGM Niterói/2023) A analista Ana configurou o pipeline MainPipe no GitLab CI/CD da PGM de Niterói. O MainPipe possui um único job, o BuildJob, que automatiza o processo de build da aplicação PGMApp. O build de PGMApp é realizado em um container Docker e demanda a execução de um banco de dados em outro container Docker. A fim de aproveitar os recursos de integração do GitLab CI/CD com o Docker, Ana especificou as imagens Docker necessárias ao build diretamente no arquivo .gitlab-ci.yml do MainPipe, no trecho associado ao BuildJob.

Logo, Ana especificou a imagem Docker para o build e a imagem Docker para o banco de dados, respectivamente, nas subseções do .gitlab-ci.yml:

a) image e services;
b) image e artifacts;
c) workflow e services;
d) workflow e artifacts;
e) artifacts e workflow.

18. (CEBRASPE/TRT 8/2022) O GitLab permite que os usuários executem verificações de segurança para garantir que as imagens do Docker não tenham nenhuma vulnerabilidade no ambiente. Essa funcionalidade é disponibilizada pelo recurso

a) Package Management.
b) Activity Stream.
c) Powerful Branching.
d) Container Scanning.
e) Auto DevOps.

19. (FCC/TRT 22/2022) Considere o recorte de um arquivo .gitlab-ci.yml que traz algumas opções de configuração de um pipeline GitLab CI/CD, nas seções que são separadas pelas global keywords.

stages:

02648216103 - HENRIQUE AMORIM BARRETO

```
  - sync
  - prepare
  - build-images

..l..:
  interruptible: true
  timeout: 90m

workflow:
# regras

variables:
# variáveis

..II.. :
   - local: .gitlab/ci/*.gitlab-ci.yml
   - remote: 'https://gitlab.com/gitlab-org/frontend/-/merge_request_pipelines.yml'
```

As global keywords I e II correspondem, correta e respectivamente, a

a) tags e directories
b) dependencies e environment
c) dependencies e pages
d) default e include
e) default e directories

20. (FCC/TJ CE/2022) Para usar o GitLab CI/CD são necessários os seguintes passos iniciais:

– Certificar-se que haja runners disponíveis para executar os jobs. Se não houver um runner, instalar o GitLab Runner e registrar um runner para a instância, projeto ou grupo.
– Criar um arquivo I na raiz do repositório. Este arquivo é o local onde se definem os jobs CI/CD.
– Quando for feito o commit deste arquivo no repositório, o runner executará os jobs.

Preenche corretamente a lacuna I:

a) .gitlab-cd.yaml
b) .gitlab-cd.yml
c) .gitlab-cicd.yaml
d) .gitlab-cicd.yml
e) .gitlab-ci.yml

GABARITO

1. Letra A
2. Letra C
3. Certo
4. Letra B
5. Letra C
6. Letra C
7. Letra A
8. Certo
9. Errado
10. Letra D
11. Errado
12. Certo
13. Certo
14. Letra D
15. Letra A
16. Letra A
17. Letra A
18. Letra D
19. Letra D
20. Letra E

02648216103 - HENRIQUE AMORIM BARRETO

ESSA LEI TODO MUNDO CONHECE:
PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.

1. 

Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. 

Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. 

Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. 

Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. 

Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. 

Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. 

Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. 

O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.